



COURSE SLIDES

Wireshark Network Analysis Masterclass

Course Code: C1123

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v5.0 · 5 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

PhD — specialises in networking, cyber security, cloud and AI.

DELIVERS

Courses on network analysis, cyber security, cloud and AI for Tertiary Infotech Academy.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

1 Your name and organisation / role.

2 Your experience with Wireshark or network troubleshooting (if any).

3 A slow or broken network problem you would like to be able to explain with packets.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

5

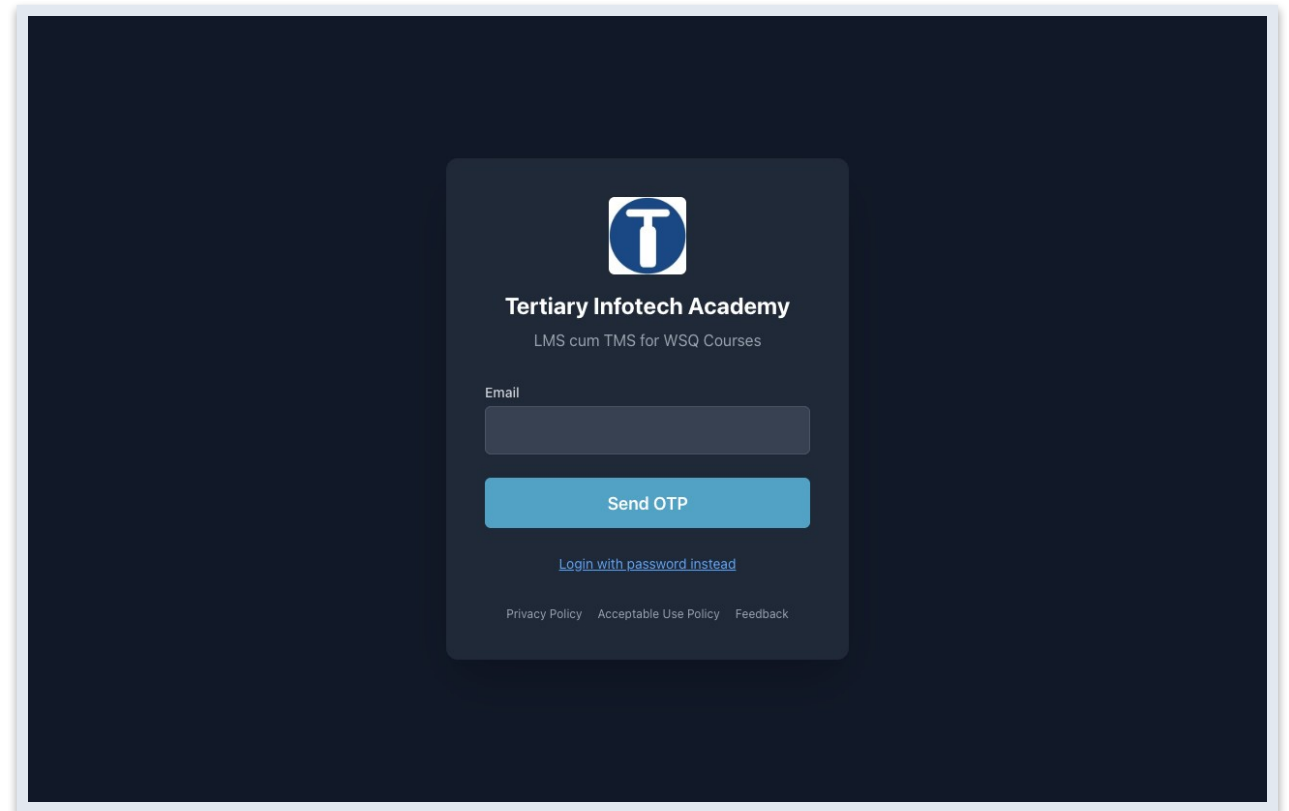
Be punctual; return from breaks on time.

6

Learn by doing — try every hands-on lab.

Download Course Material

- Go to lms-tms.tertiaryinfotech.com.
- Enter your registered email and click Send OTP.
- Key in the one-time password from your inbox to log in.
- Open this course to download the slides, Learner Guide and Lesson Plan.
- Keep them open during class — refer to them as you work through each lab.



<https://lms-tms.tertiaryinfotech.com>

SCHEDULE

Lesson Plan — 4 Days, 8 hours/day (Days 1–2)

Day 1

- Day 1 — Capture and analyst workflow
 - Topic 1: Introduction to Network Analysis and Wireshark (Lab 1)
 - Topic 2: Capture Methods and Capture Filters (Lab 2)
 - Topic 3: Global Preferences and Troubleshooting Profiles (Lab 3)
 - Topic 4: Navigation and Coloring Techniques (Lab 4)

Day 2

- Day 2 — Time, statistics, filters and DNS
 - Topic 5: Time Values and Delay Types (Lab 5)
 - Topic 6: Trace Statistics and VoIP Overview (Lab 6)
 - Topic 7: Display Filters (Lab 7)
 - Topic 8: TCP/IP Communications and Resolution (Lab 8)
 - Topic 9: DNS Traffic Analysis (Lab 9)

SCHEDULE

Lesson Plan — 4 Days, 8 hours/day (Days 3–4)

Day 3

- Day 3 — IP and transport diagnostics
 - Topic 10: ARP Traffic Analysis (Lab 10)
 - Topic 11: IPv4 Traffic Analysis (Lab 11)
 - Topic 12: ICMP Traffic Analysis (Lab 12)
 - Topic 13: UDP Traffic Analysis (Lab 13)
 - Topic 14: TCP Protocol Analysis (Lab 14)

Day 4

- Day 4 — Graphs, applications and reporting
 - Topic 15: Traffic Graphs (Lab 15)
 - Topic 16: HTTP and HTTP/2 Analysis (Lab 16)
 - Topic 17: TLS-Encrypted Traffic Analysis (Lab 17)
 - Topic 18: Ten Troubleshooting Steps and Reporting (Lab 18)
- Daily timing
 - 9:30am–6:30pm · 1-hour lunch · tea breaks within training time

Learning Outcomes

1

Capture & profile

Plan scoped captures and configure a reproducible analyst profile.

2

Navigate & time

Navigate, filter, summarize and time network exchanges.

3

Core protocols

Interpret DNS, ARP, IPv4, ICMP and UDP evidence.

4

TCP & applications

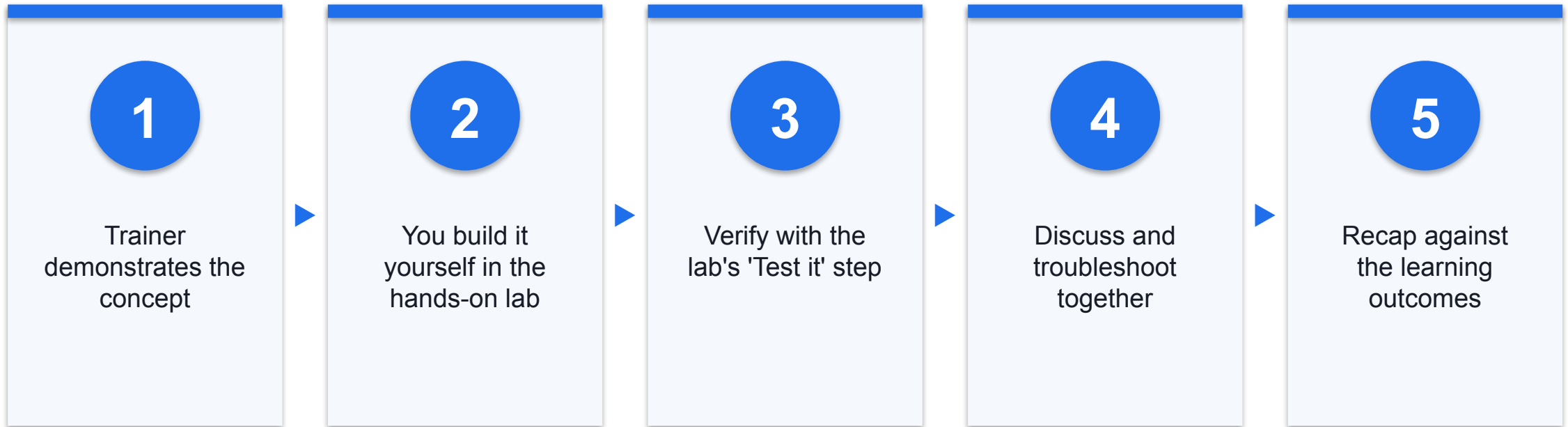
Diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

5

Evidence reporting

Report observed facts, hypotheses and next actions with defensible evidence.

How You'll Learn





CORE CONCEPTS

Network Analysis Fundamentals

What is Wireshark?

1

Packet analyzer

Captures packets and presents every field it can decode — a measuring device for the network.

2

Free and open source

Cross-platform (Windows, macOS, Linux) with TShark as its command-line twin.

3

3,000+ protocols

Dissectors decode Ethernet to HTTP/2, DNS, TLS, SIP/RTP and more.

4

Evidence, not opinion

Every claim cites a frame number, a filter and a measurement.

Who Uses Wireshark — and Why

1

Network administrators

Troubleshoot slow and failing connections.

2

Security analysts

Examine suspicious conversations and indicators.

3

QA and developers

Verify and debug protocol implementations.

4

Support engineers

Prove where a delay or failure actually occurs.

5

Learners

See how network protocols really behave on the wire.

6

Incident responders

Preserve packet evidence for a report.

THE ANALYST MINDSET

Capture the evidence. Explain the uncertainty.

A trace is a measurement at one observation point — this course teaches you to say exactly what it proves and what it does not.

The Four Days at a Glance

Capture & navigate

- Place the capture point correctly
- Build a reproducible analyst profile
- Colour, time and filter efficiently

Protocols & transport

- DNS, ARP, IPv4, ICMP, UDP evidence
- TCP handshake, loss, retransmission
- Zero window and recovery

Applications & reporting

- I/O, RTT and Stevens graphs
- HTTP outcomes, HTTP/2 and TLS
- An evidence-led incident report

How Every Lab Progresses

1

Open the lab's own synthetic capture — no live network traffic is needed.

2

Apply the lab filter and locate the frames that answer the question.

3

Record frame numbers, measurements and one limitation in outputs/findings.md.

4

Run scripts/verify.py — it confirms the capture facts your findings rely on.

TOPIC 01

Introduction to Network Analysis and Wireshark

Establish a trace baseline

KEY CONCEPTS

Key Concepts — Introduction to Network Analysis and Wireshark

1

A capture is a measurement at one observation point, not a complete network history.

2

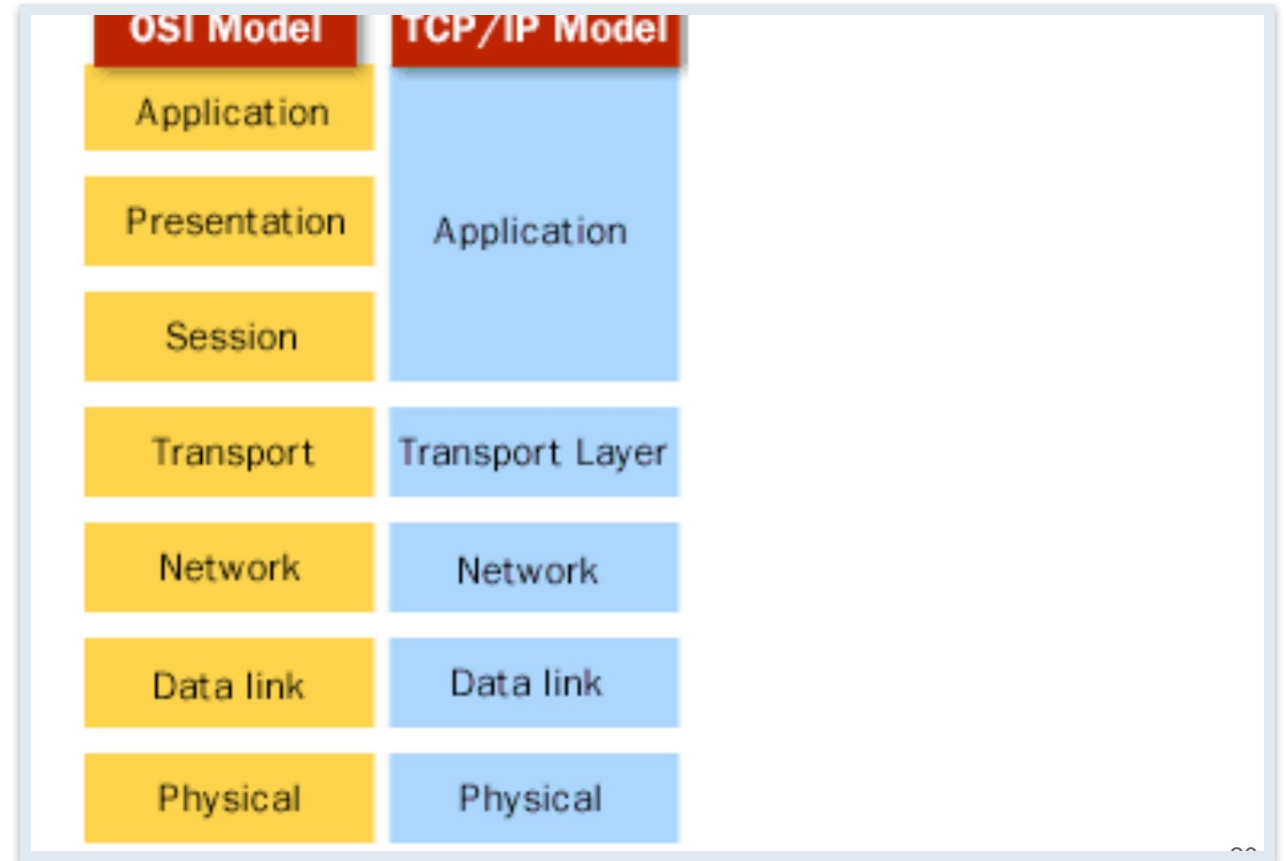
Packet list, protocol details and bytes connect a summary to the underlying evidence.

3

Record interface, timestamp precision, dropped packets and capture scope before interpreting a trace.

The TCP/IP and OSI Models

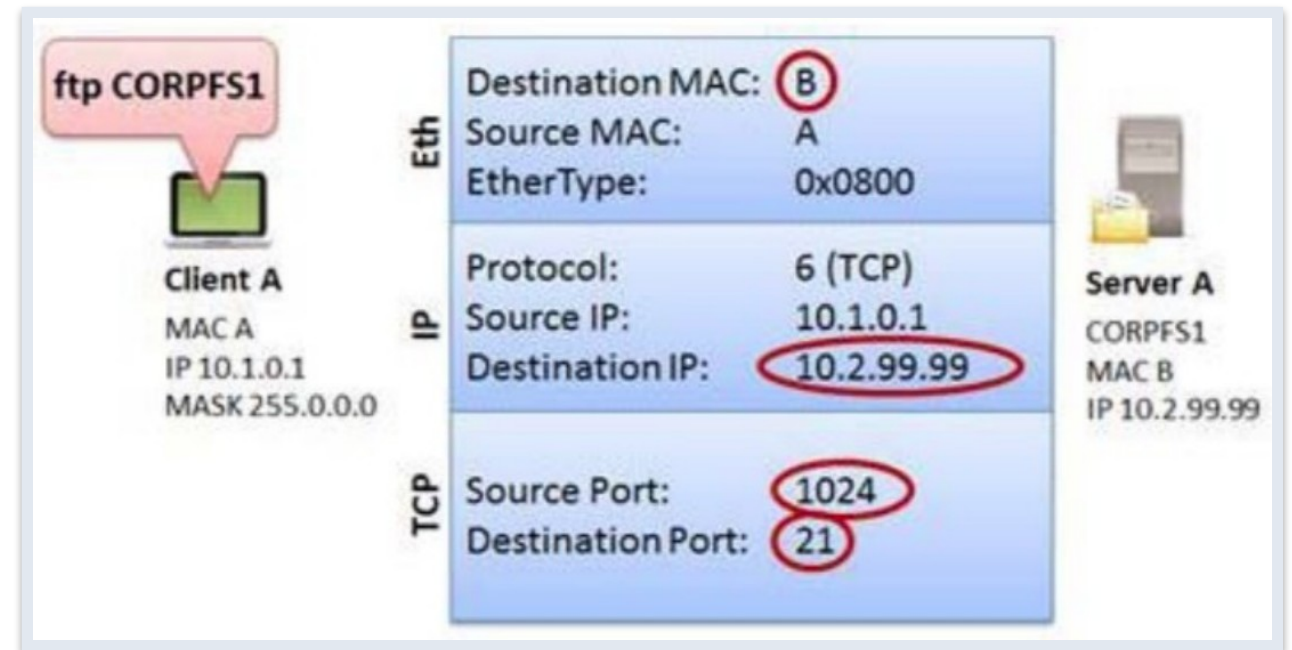
- The OSI model has seven layers; TCP/IP folds them into four.
- Wireshark's Packet Details pane is ordered the same way — frame, Ethernet, IP, TCP/UDP, application.
- Locate each fault at a layer before you choose a filter.



OSI layers mapped to the TCP/IP model

Building a Packet

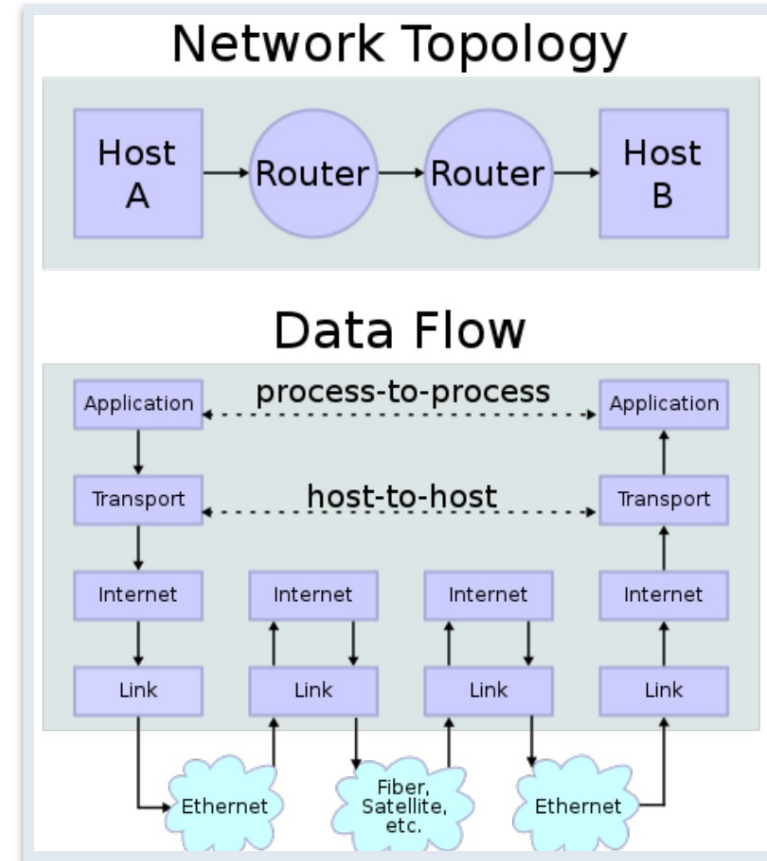
- Each layer wraps the one above: an application payload gains TCP, IP and Ethernet headers.
- MAC addresses change at every hop; IP addresses stay end-to-end (unless NAT rewrites them).
- Ports identify the conversation on each host.



Encapsulation of an FTP request from client A to server A

Data Flow Through the Network

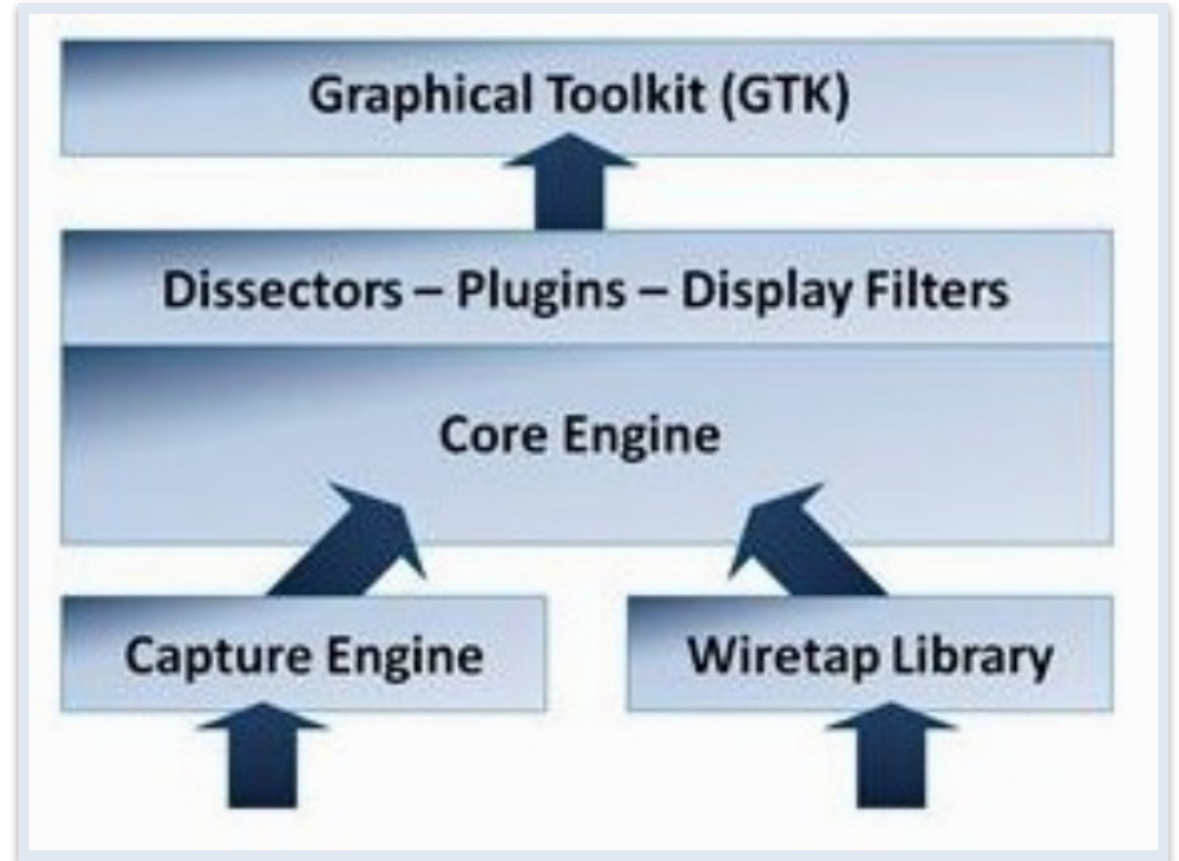
- Data passes down the sender's stack, across switches and routers, and up the receiver's stack.
- Switches forward on MAC; routers forward on IP.
- Where you capture decides which hops you can see.



Process-to-process, host-to-host and hop-by-hop delivery

Inside Wireshark

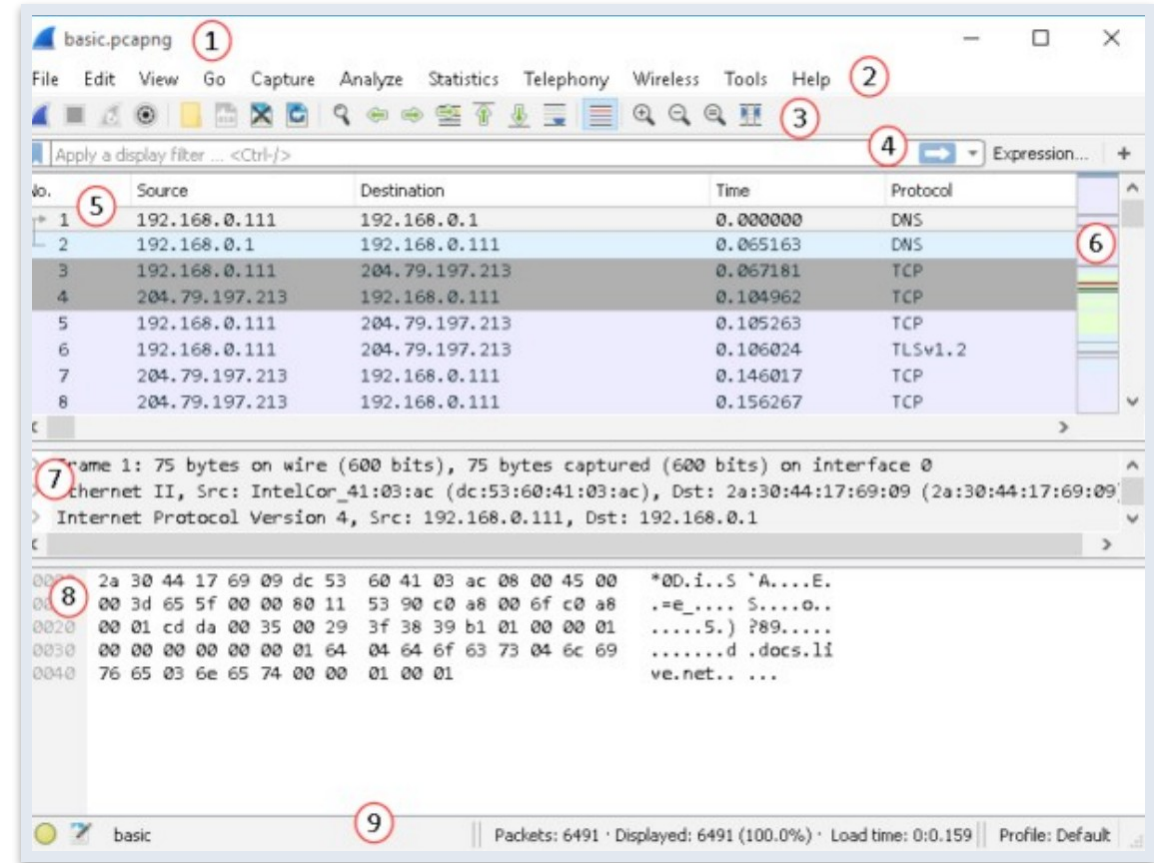
- The capture engine collects packets through libpcap / Npcap.
- Wiretap reads and writes trace formats: pcap, pcapng, snoop and more.
- Dissectors decode each protocol; display filters operate on the dissected fields.



Packet processing elements (the GUI is Qt-based in current releases)

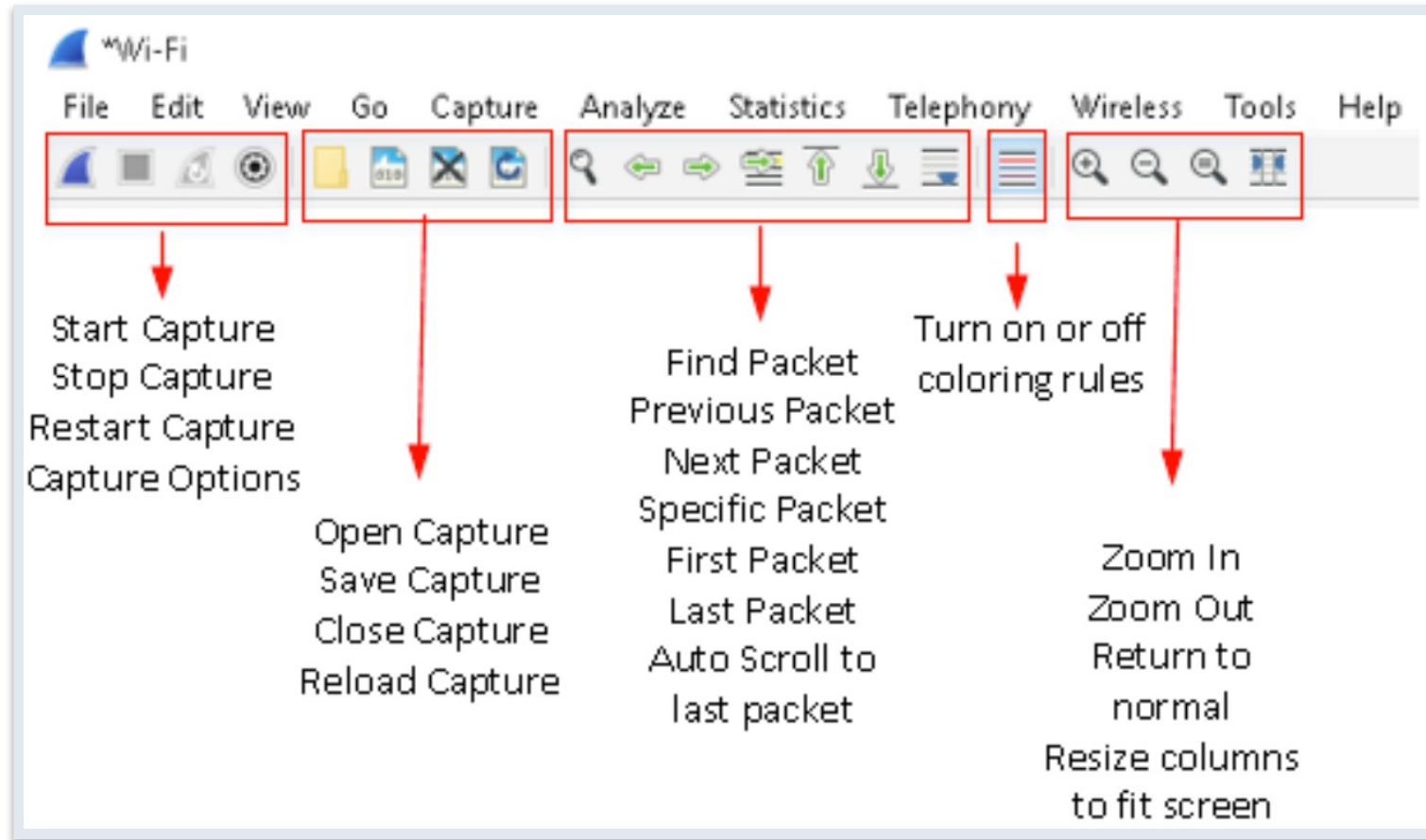
The Wireshark Window

- Packet List — one summary row per frame.
- Packet Details — the decoded protocol tree.
- Packet Bytes — the raw hex for the selected field.
- Status bar — packet counts, profile and file information.



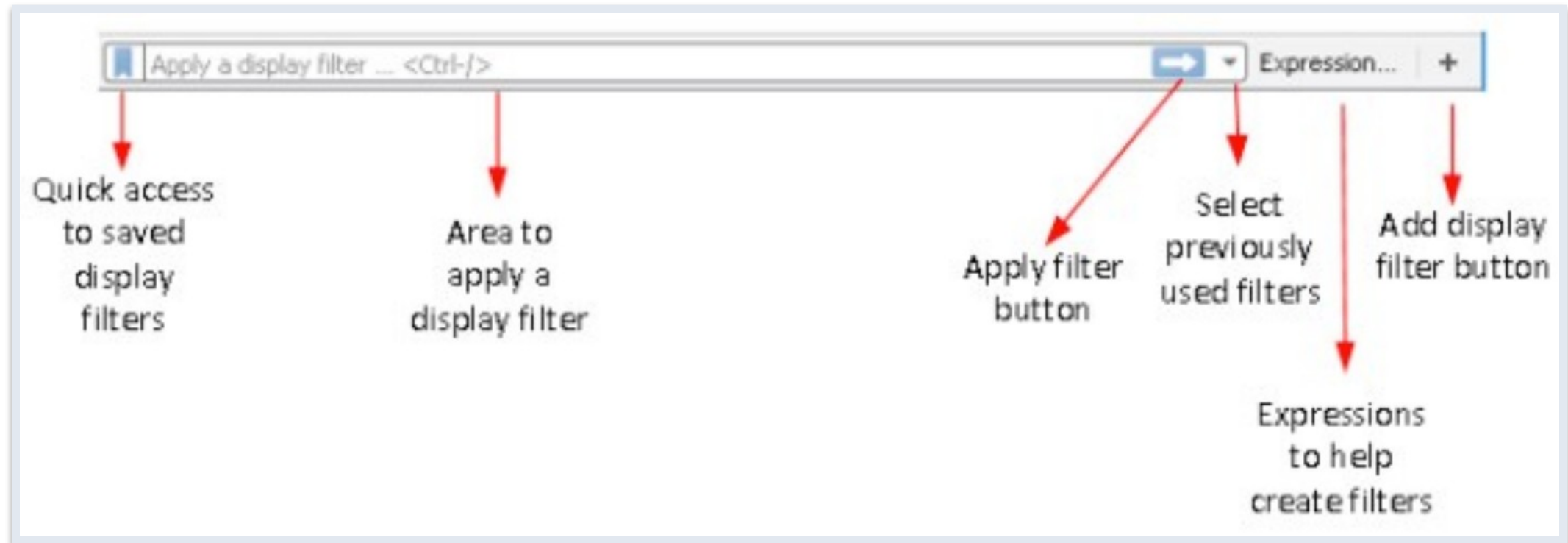
The three main panes plus toolbars and status bar

The Main Toolbar



Start/stop capture, open/save, find and go-to packet, colourise and zoom controls

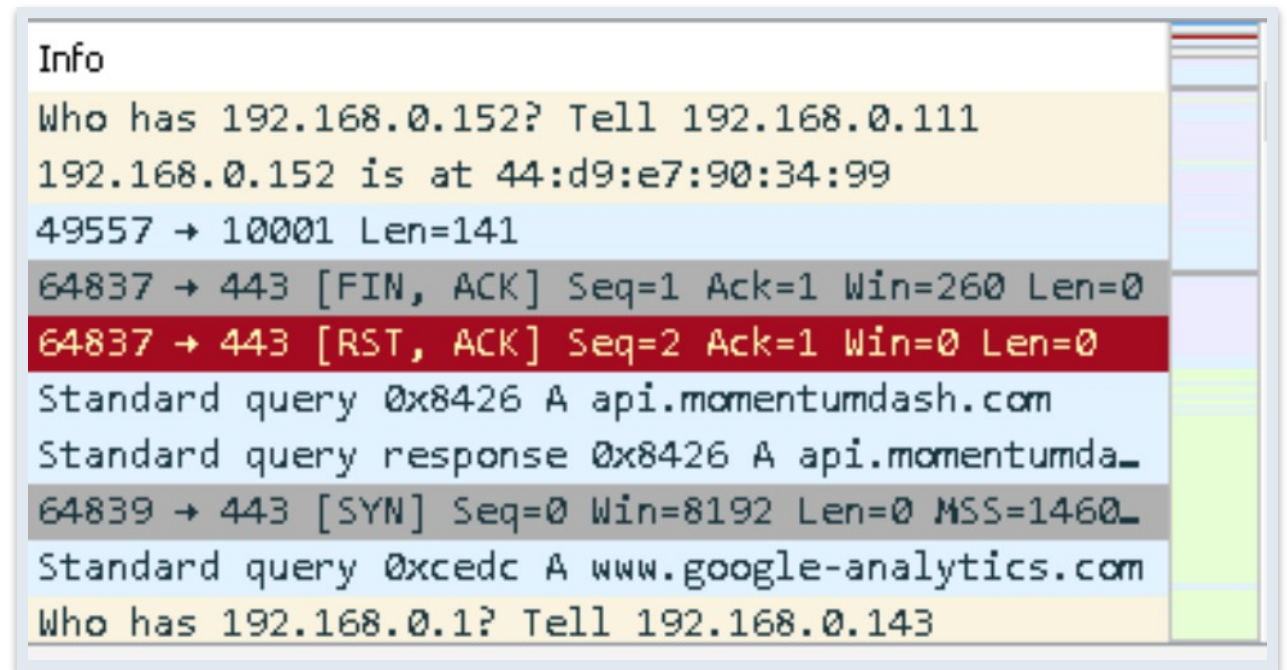
The Display Filter Toolbar



Green = valid expression, red = invalid; bookmark saved filters and add filter buttons

Status Bar and Intelligent Scrollbar

- The status bar shows packets captured vs. displayed and the active profile.
- The intelligent scrollbar mirrors packet colouring — red/black bands reveal resets, aborts or low-TTL packets at a glance.



Coloured scrollbar bands mark problem packets

Establish a trace baseline

LAB 1

A capture is a measurement at one observation point, not a complete network history.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 1

arp					
No.	Time	Source	Destination	Protocol	Info
1	0.000000	02:00:00:00:00:10	ff:ff:ff:ff:ff:ff	ARP	Who has 192.0.2.20? Tell 192.0.2.10
2	0.010000	02:00:00:00:00:20	02:00:00:00:00:10	ARP	192.0.2.20 is at 02:00:00:00:00:20
branch-office.pcap Displayed: 2					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Establish a trace baseline

STEP 1 OF 5

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Establish a trace baseline

STEP 2 OF 5



2

Open data/branch-office.pcap with File > Open. Clear the display filter. Record the packet count from the status bar and the capture duration from Statistics > Capture File Properties.

Establish a trace baseline

STEP 3 OF 5

3

Select frame 1. Expand Ethernet II and Address Resolution Protocol in Packet Details. Click the sender IP field and observe the corresponding bytes.

Establish a trace baseline

STEP 4 OF 5

4

Apply arp in the display filter toolbar. Compare request and reply sender/target IP and MAC addresses. Save the two frame numbers in outputs/findings.md.

Establish a trace baseline

STEP 5 OF 5

5

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Establish a trace baseline

Test it

The supplied arp expression matches 2 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

TOPIC RECAP

Recap — Introduction to Network Analysis and Wireshark

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A capture is a measurement at one observation point, not a complete network history.

3

Packet list, protocol details and bytes connect a summary to the underlying evidence.

4

Record interface, timestamp precision, dropped packets and capture scope before interpreting a trace.

Tea Break

15 minutes

TOPIC 02

Capture Methods and Capture Filters

Plan capture placement

KEY CONCEPTS

Key Concepts — Capture Methods and Capture Filters

1

A switched access port normally observes its own traffic and broadcasts; SPAN or TAP placement changes visibility.

2

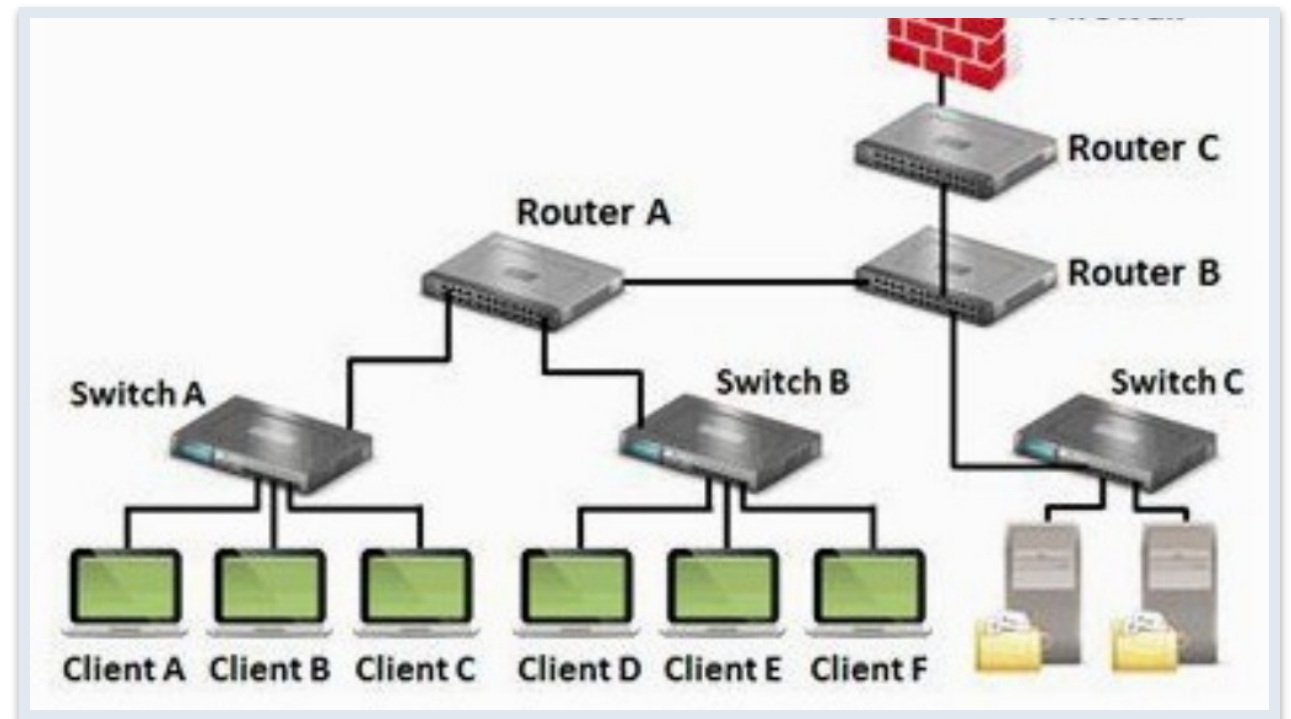
Capture filters use libpcap syntax before packets are stored; display filters hide or show stored packets.

3

Bound file size and duration; check capture drops before attributing missing packets to network loss.

Where to Tap Into the Network

- Client A complains about reaching the servers — where do you put the analyzer?
- Capture as close as possible to the complaining host first, then move toward the servers.
- Each point sees only the traffic that physically passes it.



Choose the observation point before you capture

What a Switch Port Can See

Switch port (default)

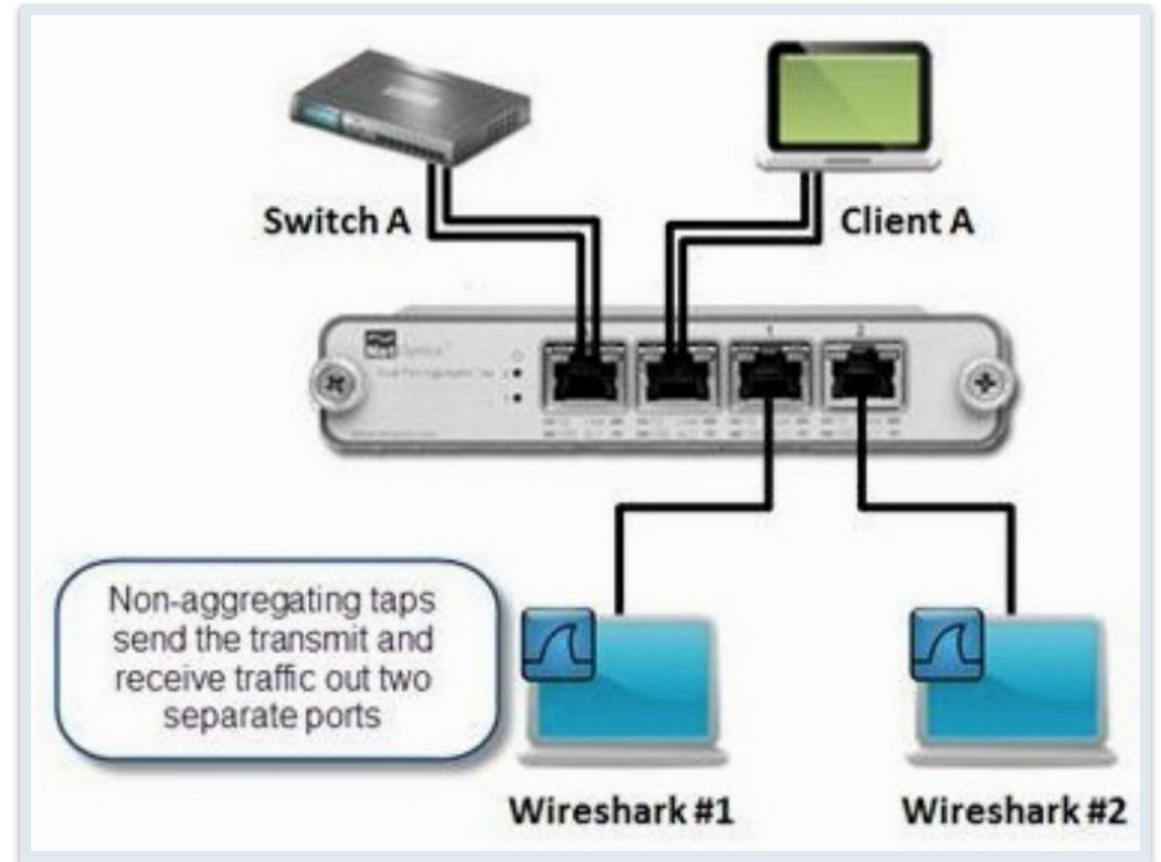
- Broadcast traffic
- Multicast traffic (if forwarded)
- Traffic to and from your own host
- Not other hosts' unicast traffic

Ways to see more

- Hub in half-duplex segment (lab only)
- Network TAP — passive, in-line
- SPAN / port mirroring on the switch
- Capture on the target host itself

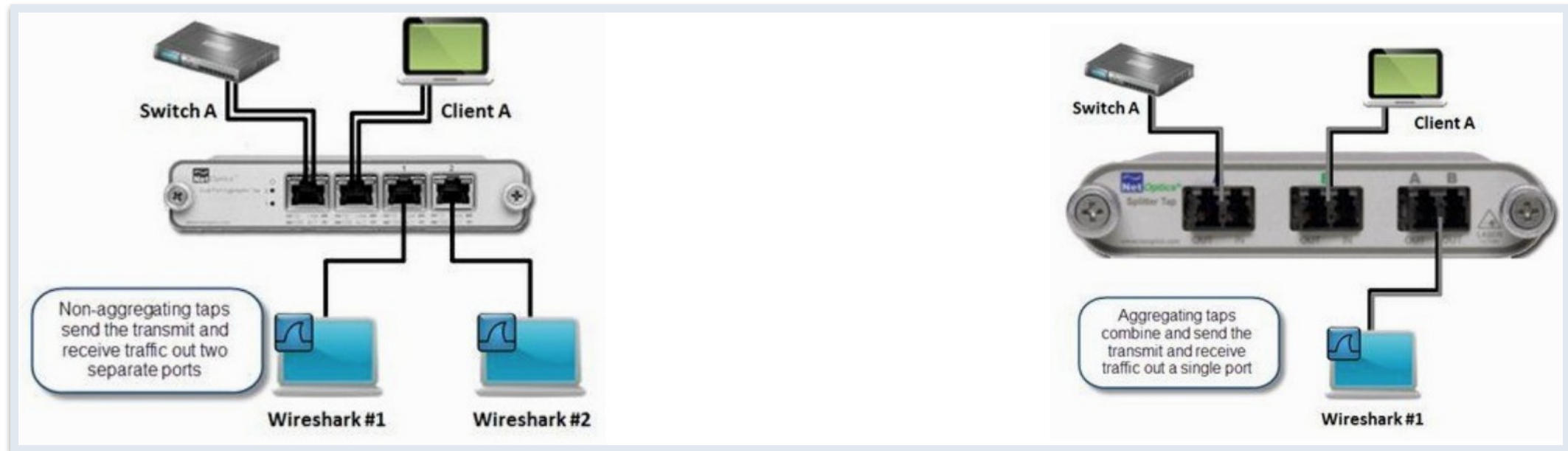
Test Access Points (TAPs)

- A TAP sits in-line and passively copies traffic to the analyzer.
- Full-duplex TAPs forward both directions — and can pass physical-layer errors.
- Aggregating TAPs merge both directions onto one monitor port.



A non-aggregating TAP with two monitor ports

Aggregating vs Non-aggregating TAPs

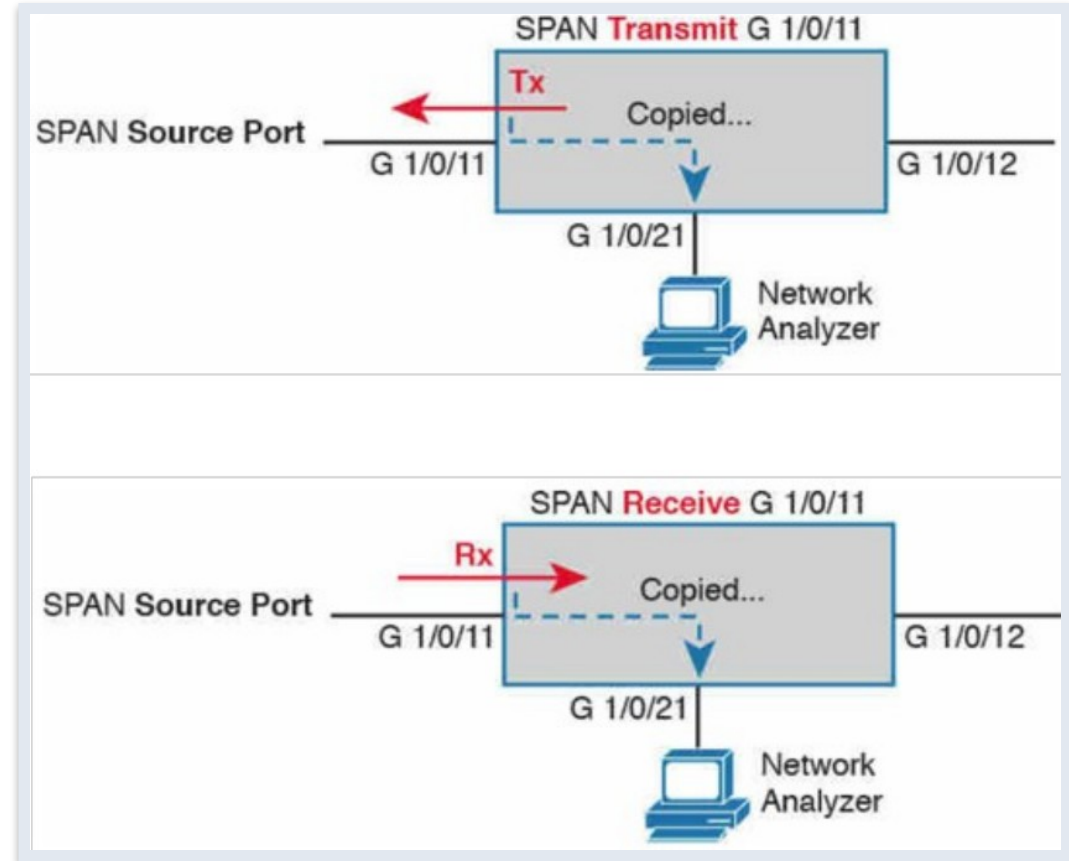


Non-aggregating: one port per direction. Aggregating: both directions combined on one port — can oversubscribe at high load

SPAN

SPAN / Port Mirroring

- The switch copies traffic from source ports or VLANs to a destination port.
- Mirrored traffic can drop under load and normally excludes errored frames.
- Example (Cisco): monitor session 1 source interface g1/0/11 both.



SPAN copies transmitted and received frames to the analyzer port

Wireless Capture

- Promiscuous mode on Wi-Fi usually shows only your own traffic.
- Monitor mode captures 802.11 frames from all stations on a channel — needs a supporting adapter/driver.
- Decrypting WPA2 traffic requires the passphrase and the 4-way handshake.



Wireless capture depends on the adapter mode

Capture Filters vs Display Filters

Capture filter (BPF)

- Applied before packets are stored
- Syntax: host 10.1.1.5 and tcp port 80
- Discarded packets cannot be recovered
- Use to limit file size on busy links

Display filter

- Applied to packets already captured
- Syntax: ip.addr == 10.1.1.5 && tcp.port == 80
- Clear it to see everything again
- Use for analysis and evidence

Plan capture placement

LAB 2

A switched access port normally observes its own traffic and broadcasts; SPAN or TAP placement changes visibility.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 2

tcp.port == 80					
No.	Time	Source	Destination	Protocol	Info
18	1.040000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 W...
19	1.070000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0...
20	1.080000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=1 Ack=1 Win=1048576 Len=0
21	1.090000	192.0.2.10	192.0.2.20	HTTP	GET /health HTTP/1.1
22	1.100000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [ACK] Seq=1 Ack=71 Win=1048576 Len=0
23	1.140000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
24	1.150000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=71 Ack=71 Win=8222720 Len=0
25	1.160000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [FIN, ACK] Seq=71 Ack=71 Win=1048576 L...
26	1.170000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=71 Ack=72 Win=1048576 Len=0
27	1.180000	192.0.2.10	192.0.2.20	TCP	51002 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 W...
... 27 more matching packets					
branch-office.pcap Displayed: 37					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Plan capture placement

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Plan capture placement

STEP 2 OF 6

2

Open assets/topology.md. Draw a sensor on the client access port, on a mirrored uplink and beside the server. State which traffic each sees.

Plan capture placement

STEP 3 OF 6

3

Open Capture > Options and inspect the interface list without starting a capture. Enter tcp port 80 in the capture-filter box and check syntax feedback. Cancel the dialog.

Plan capture placement

STEP 4 OF 6

4

Open data/branch-office.pcap. Enter `tcp.port == 80` in the DISPLAY toolbar. Compare the stored total with the displayed total; explain why clearing it restores hidden packets.

Plan capture placement

STEP 5 OF 6

5

Complete assets/capture-plan.md with scope, interface, duration, ring-buffer limit and permission owner. Wireless monitor mode requires compatible adapter/driver; USB capture requires platform-specific support. Remote capture should use authorised SSH/extcap, not obsolete unauthenticated RPC instructions.

Plan capture placement

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Plan capture placement

Test it

The supplied `tcp.port == 80` expression matches 37 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Capture Methods and Capture Filters

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A switched access port normally observes its own traffic and broadcasts; SPAN or TAP placement changes visibility.

3

Capture filters use libpcap syntax before packets are stored; display filters hide or show stored packets.

4

Bound file size and duration; check capture drops before attributing missing packets to network loss.

Lunch Break

1 hour

TOPIC 03

Global Preferences and Troubleshooting Profiles

Create an analyst profile

KEY CONCEPTS

Key Concepts — Global Preferences and Troubleshooting Profiles

1

A profile bundles reproducible columns, coloring rules and protocol settings.

2

Name resolution can obscure numeric evidence and generate additional traffic; document the setting.

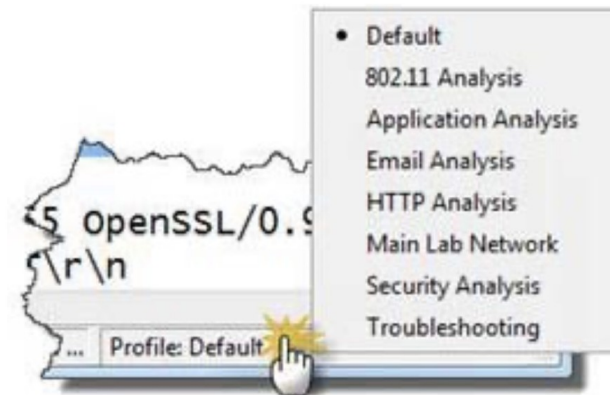
3

Protocol heuristics and TCP analysis preferences change interpretation, not the stored bytes.

Configuration Profiles

- A profile bundles preferences, columns, colouring rules and filter buttons.
- Switch profiles from the status bar or Edit > Configuration Profiles.
- Create task profiles: Troubleshooting, Security, VoIP, Wireless.

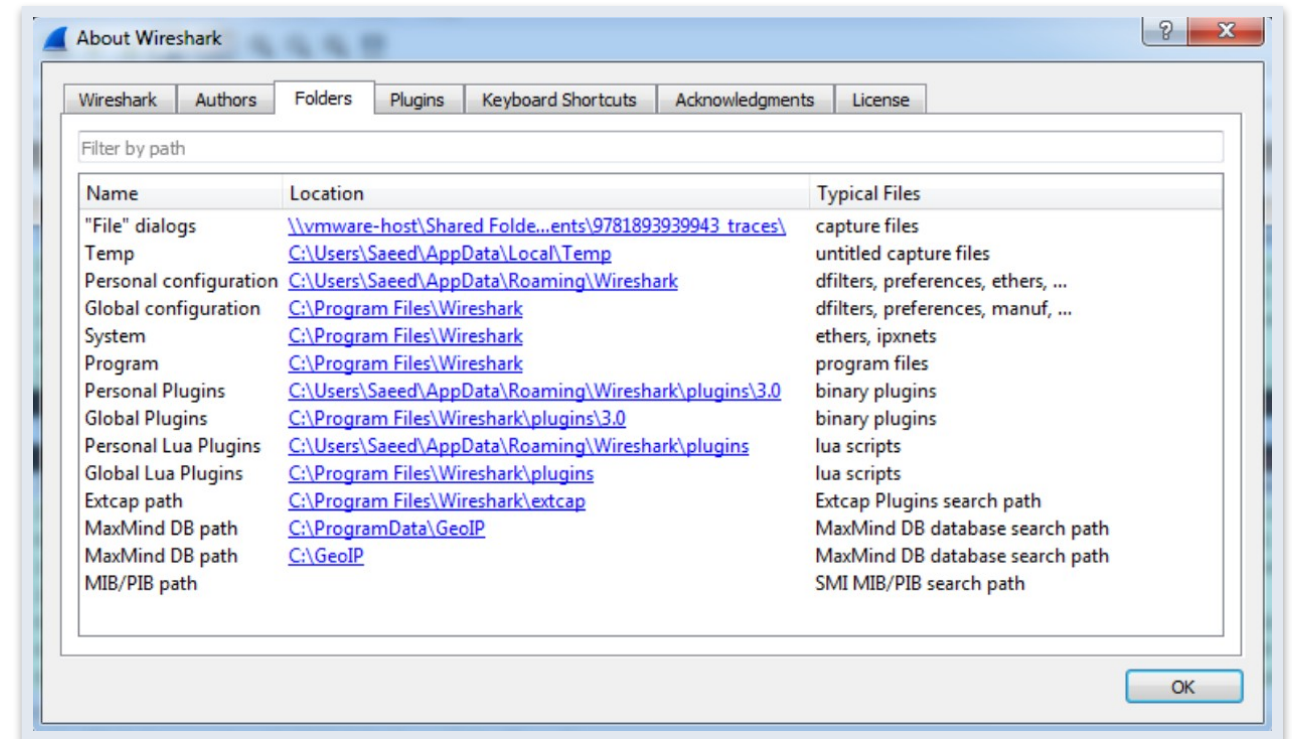
Right click on the Profile area in the Status bar or select **Edit | Configuration Profiles | New** to create a new profile.



Right-click the Profile area in the status bar to switch or create

Where Settings Live

- Help > About Wireshark > Folders shows personal and global configuration paths.
- Copy a profile folder to share it with your team.
- Keep a clean Default profile untouched.



The Folders tab lists personal configuration locations

Preferences That Change Interpretation

Name resolution

- Off by default for evidence
- MAC/transport names are local look-ups
- Network names cause extra DNS traffic

Columns

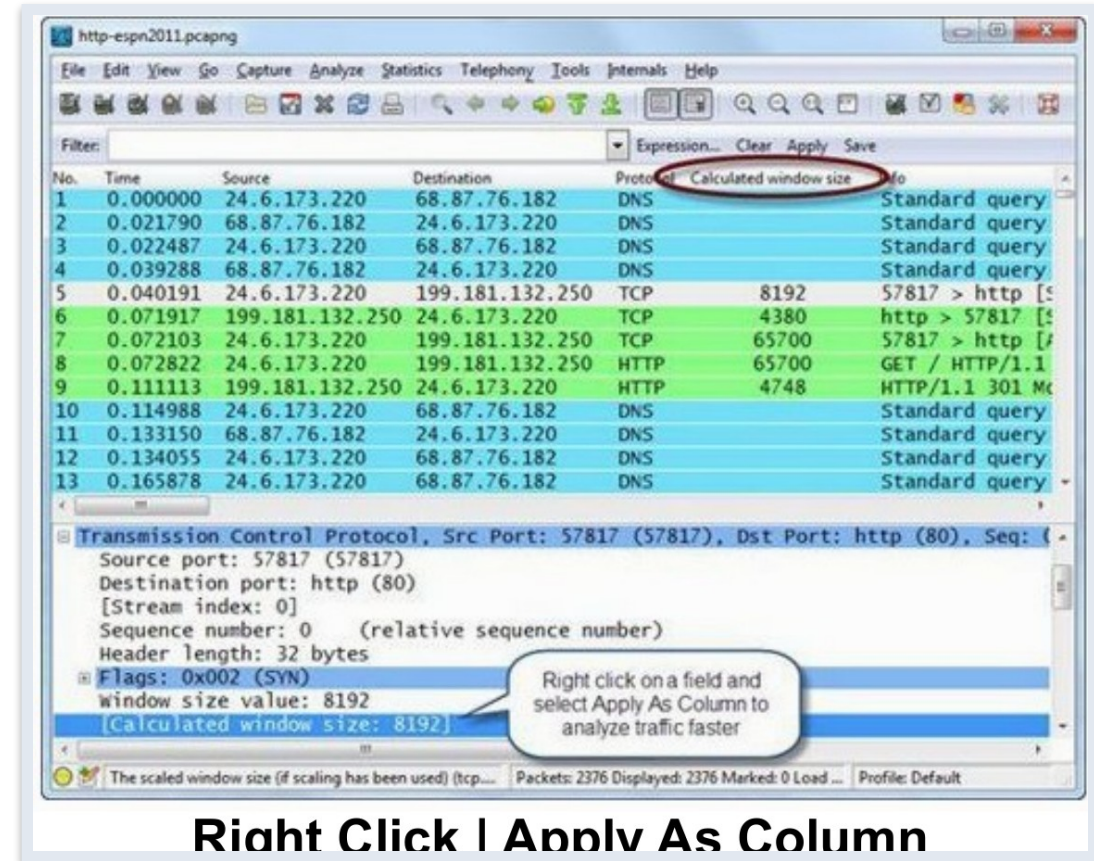
- Add fields as columns (Apply as Column)
- e.g. tcp.time_delta, http.host
- Save them in the profile

Protocol settings

- TCP: calculate conversation timestamps
- TCP: relative sequence numbers
- TLS: key log file per lab

Apply as Column

- Right-click any field in Packet Details > Apply as Column.
- Sort the new column to find extremes — largest delay, largest window, error codes.
- Columns are saved in the active profile.



Right Click | Apply As Column

Right-click | Apply As Column

Create an analyst profile

LAB 3

A profile bundles reproducible columns, coloring rules and protocol settings.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 3

dns					
No.	Time	Source	Destination	Protocol	Info
3	0.020000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0065 A portal.example.test
4	0.040000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0065 A portal.example.t...
5	0.050000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0066 A missing.example.test
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
7	0.080000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0067 A slow.example.test
8	0.880000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0067 A slow.example.tes...
branch-office.pcap Displayed: 6					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Create an analyst profile

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Create an analyst profile

STEP 2 OF 6

2

Create a profile named C1123-Analyst using Edit > Configuration Profiles. Keep the original profile available.

Create an analyst profile

STEP 3 OF 6

3

Open data/branch-office.pcap. Disable network name resolution in View > Name Resolution. Record numeric client, server and resolver IP addresses.

Create an analyst profile

STEP 4 OF 6

4

Select a TCP packet. Expand TCP and right-click Stream index > Apply as Column. Add frame.time_delta_displayed as a custom column using Preferences > Appearance > Columns.

Create an analyst profile

STEP 5 OF 6

5

Apply dns and inspect the profile name and displayed columns. Copy the profile folder path from Help > About Wireshark > Folders into outputs/findings.md. Export or copy your profile to outputs/profile/ without overwriting a colleague profile.

Create an analyst profile

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Create an analyst profile

Test it

The supplied dns expression matches 6 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

TOPIC RECAP

Recap — Global Preferences and Troubleshooting Profiles

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A profile bundles reproducible columns, coloring rules and protocol settings.

3

Name resolution can obscure numeric evidence and generate additional traffic; document the setting.

4

Protocol heuristics and TCP analysis preferences change interpretation, not the stored bytes.

Tea Break

15 minutes

TOPIC 04

Navigation and Coloring Techniques

Color and annotate evidence

KEY CONCEPTS

Key Concepts — Navigation and Coloring Techniques

1

Color rules are applied in order; the first matching rule wins.

2

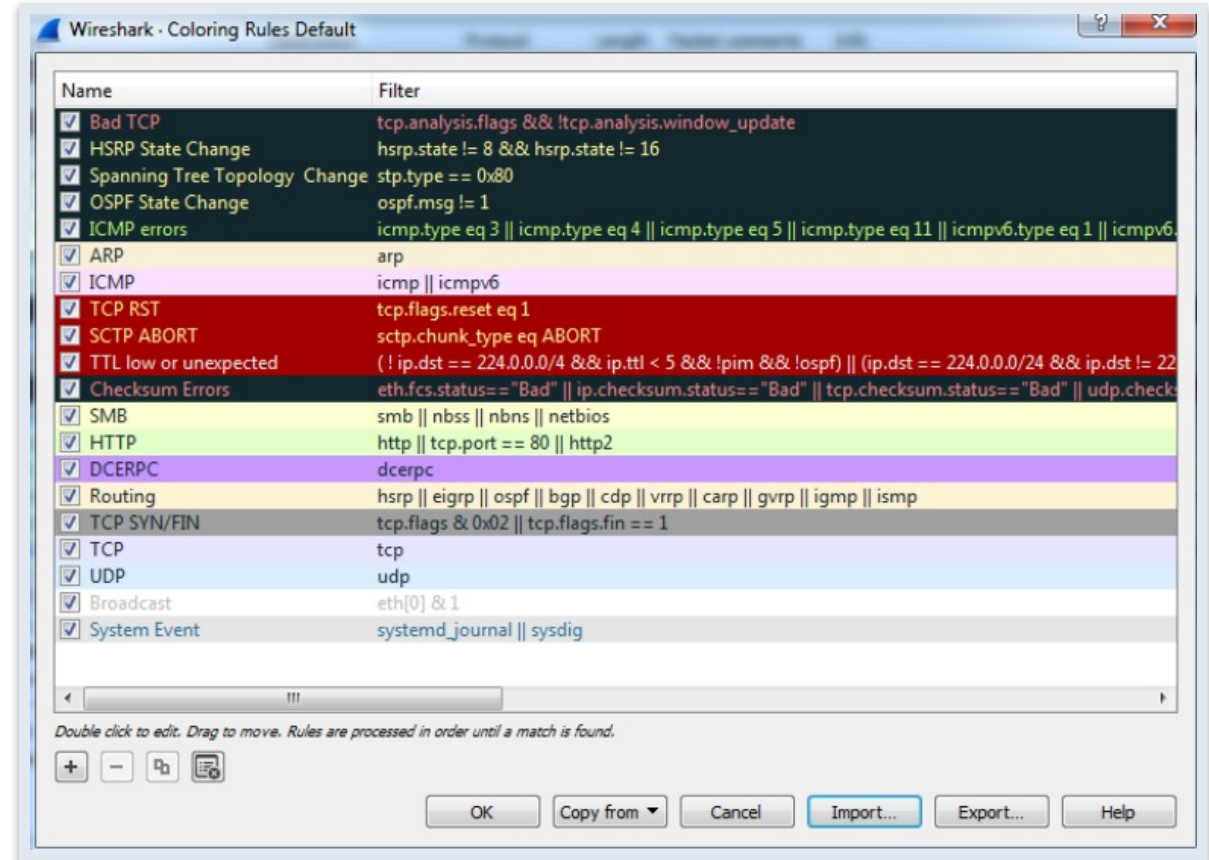
Temporary coloring is useful for a conversation; permanent rules support repeated triage.

3

Bookmarks and comments preserve the analyst path without changing the captured payload.

Colouring Rules

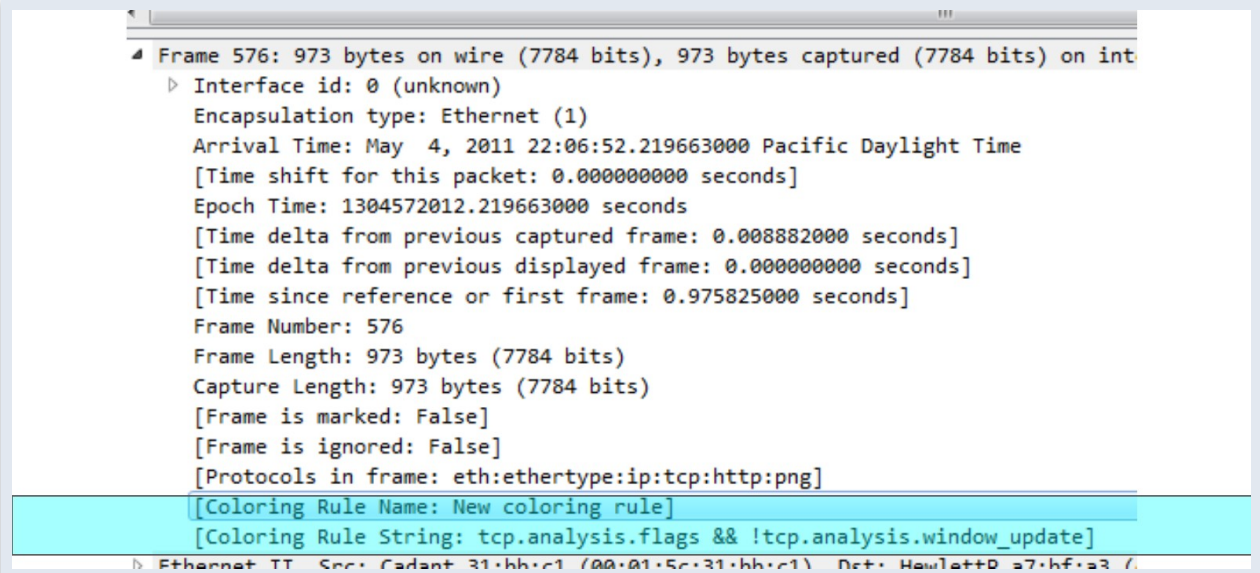
- Rules are evaluated top-down — the first match colours the packet.
- Default rules flag Bad TCP, HTTP, DNS, ARP, ICMP errors and more.
- View > Coloring Rules to edit, import or export a rule set.



The coloring rules list is processed in order

Why Is a Packet This Colour?

- Expand the Frame section of Packet Details.
- Coloring Rule Name and Coloring Rule String show which rule matched.
- Use this to explain colours in a report.

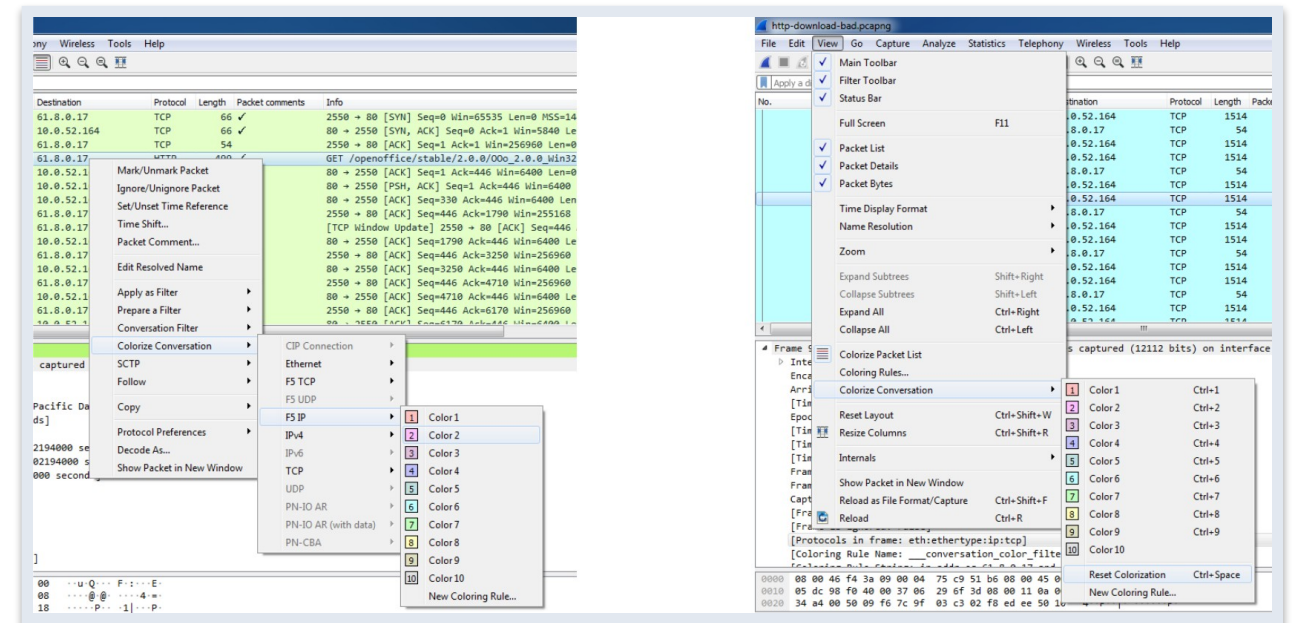


```
4 Frame 576: 973 bytes on wire (7784 bits), 973 bytes captured (7784 bits) on int
  ▸ Interface id: 0 (unknown)
    Encapsulation type: Ethernet (1)
    Arrival Time: May  4, 2011 22:06:52.219663000 Pacific Daylight Time
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1304572012.219663000 seconds
    [Time delta from previous captured frame: 0.008882000 seconds]
    [Time delta from previous displayed frame: 0.000000000 seconds]
    [Time since reference or first frame: 0.975825000 seconds]
    Frame Number: 576
    Frame Length: 973 bytes (7784 bits)
    Capture Length: 973 bytes (7784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:http:png]
    [Coloring Rule Name: New coloring rule]
    [Coloring Rule String: tcp.analysis.flags && !tcp.analysis.window_update]
  ▸ Ethernet II  Src: Cadent 31:hh:c1 (00:01:5c:31:hh:c1)  Dst: HewlettD a7:bf:a3 (/
```

Frame > Coloring Rule Name / String

Colourise a Conversation

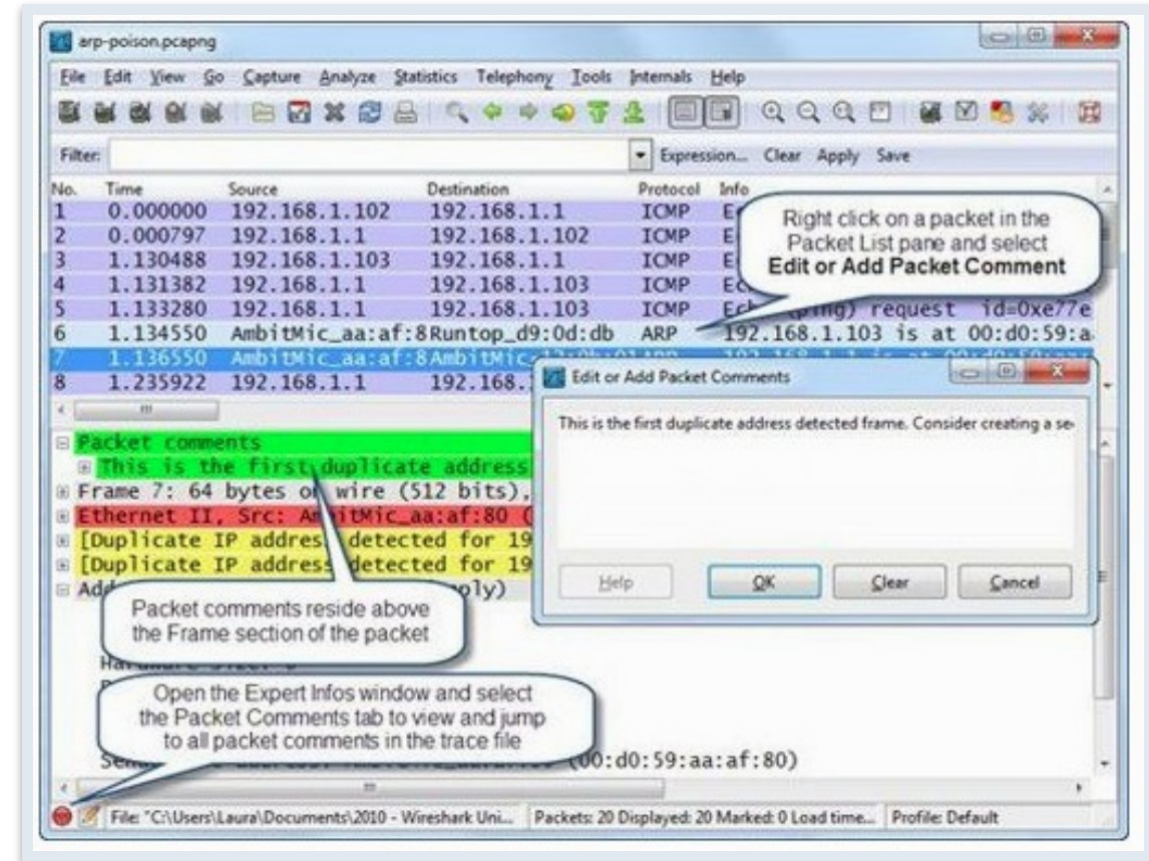
- Right-click a packet > Colorize Conversation to tag a TCP/UDP stream temporarily.
- Ten temporary colours make parallel conversations easy to tell apart.
- View > Colorize Conversation > Reset to clear.



Temporary conversation colouring

Marking and Annotating Packets

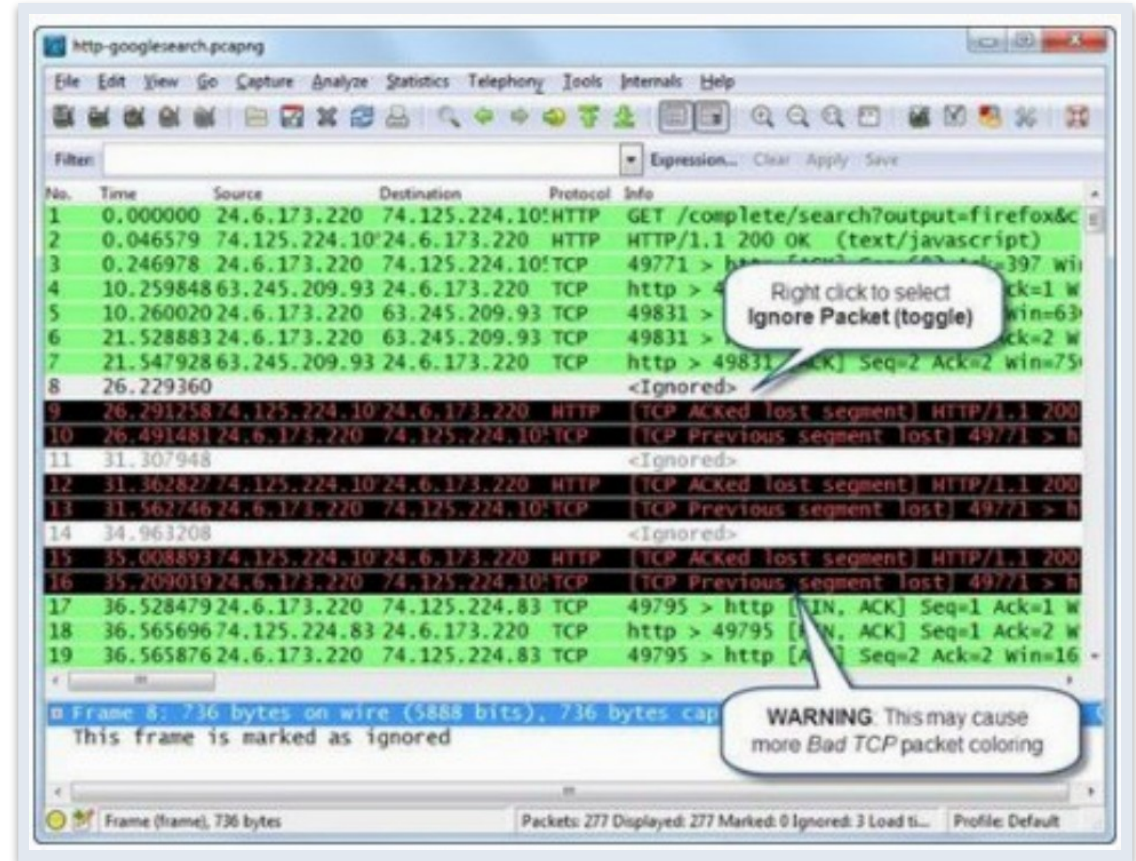
- Mark (Ctrl+M) for quick navigation — marks are lost when the file closes.
- Packet comments are saved in pcapng and travel with the evidence.
- Time Shift corrects clock offsets between captures.



Right-click | Packet Comment

Navigation Shortcuts

- Ctrl+G — go to packet number.
- Ctrl+F — find by display filter, hex or string.
- Ctrl+. / Ctrl+, — next/previous packet in the conversation.
- Ctrl+Space — collapse/expand the Packet Details tree.



Edit | Mark / Ignore packets

Color and annotate evidence

LAB 4

Color rules are applied in order; the first matching rule wins.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 4

http.response.code >= 400					
No.	Time	Source	Destination	Protocol	Info
41	2.130000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 404 Not Found (text/plain)
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Color and annotate evidence

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```


Color and annotate evidence

STEP 2 OF 6

A green circle with a white number 2 inside, indicating the second step of the process.

Open data/branch-office.pcap. Apply `http.response.code >= 400`.
Record the 404 and 500 frame numbers.

Color and annotate evidence

STEP 3 OF 6

3

Open View > Coloring Rules. Add LAB HTTP Error with expression `http.response.code >= 400`, dark text and a pale amber background. Move it above a general HTTP rule.

Color and annotate evidence

STEP 4 OF 6

4

Select the 500 response, mark it using Edit > Mark/Unmark Packet and add a packet comment describing the observed status only. Save As outputs/annotated.pcapng to preserve comments.

Color and annotate evidence

STEP 5 OF 6

5

Clear the filter. Use Edit > Find Packet with Display filter `http.response.code == 500`. Reopen the saved pcapng and confirm the comment persists.

Color and annotate evidence

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Color and annotate evidence

Test it

The supplied `http.response.code >= 400` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Navigation and Coloring Techniques

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

Color rules are applied in order; the first matching rule wins.

3

Temporary coloring is useful for a conversation; permanent rules support repeated triage.

4

Bookmarks and comments preserve the analyst path without changing the captured payload.

End of Day 1

See you tomorrow

DAY 2

Time, statistics, filters and DNS

Recap of the previous day, then today's topics and labs

TOPIC 05

Time Values and Delay Types

Separate path and server delay

Key Concepts — Time Values and Delay Types

1

Displayed delta depends on the current filter; capture delta does not.

2

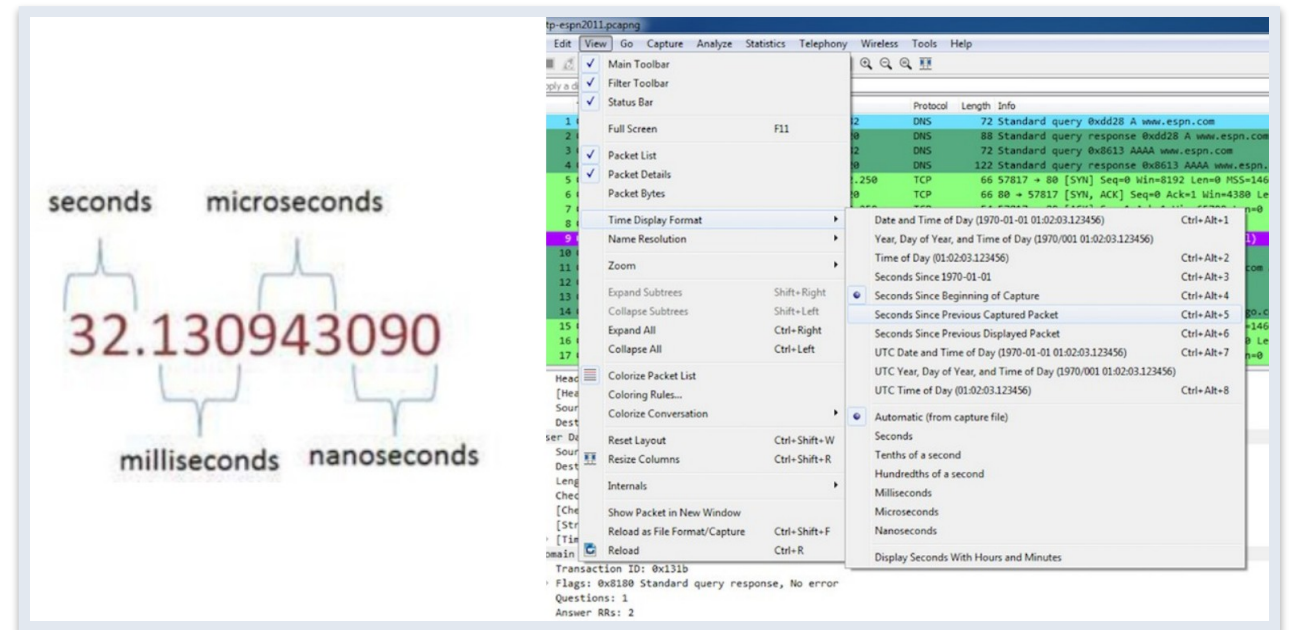
SYN to SYN/ACK gives an initial path-related sample; request to response includes application processing.

3

One-sided timestamps cannot identify exactly which intermediate device delayed or dropped a packet.

Time Display Formats

- View > Time Display Format chooses absolute, relative or delta time.
- Seconds Since Beginning of Capture — default overview.
- Seconds Since Previous Displayed Packet — finds gaps in a filtered view.



Precision depends on the capture hardware and file format

Captured vs Displayed Delta

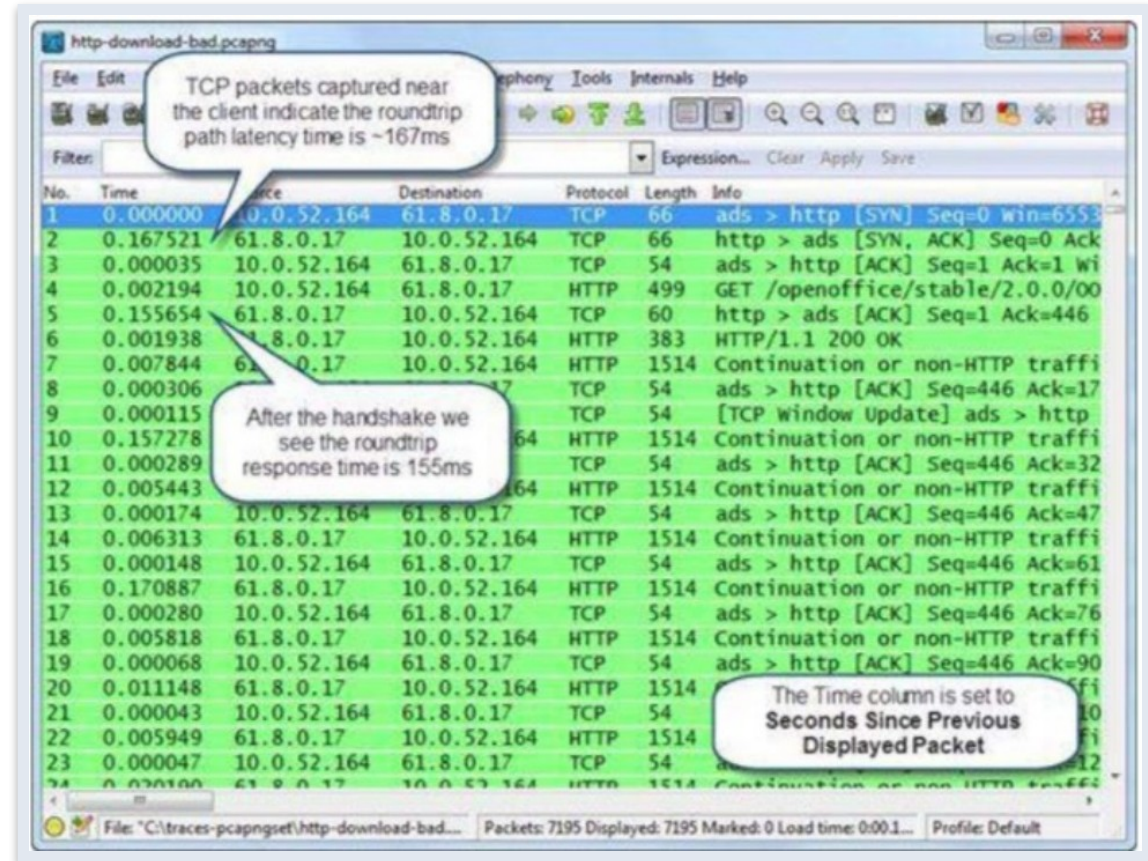
- Delta since previous CAPTURED packet ignores the filter.
- Delta since previous DISPLAYED packet changes when you change the filter.
- State which delta you used in every finding.

	Captured Packet	Displayed Packet
1	0.000000	0.000000
2	0.001000	0.001000
3	0.013000	-----
4	0.002000	0.015000
5	0.034000	-----
6	0.006000	-----
7	0.018000	0.058000
8	0.005000	0.005000

Same packets — different deltas

End-to-End Path Delay

- SYN → SYN/ACK gives the initial round-trip time (iRTT) near the client.
- Request → response adds server processing time.
- Large gaps after an ACK point to the server, not the path.



Sort a delta-time column to find the largest gaps

Three Kinds of Delay

Path delay

- SYN → SYN/ACK (iRTT)
- `tcp.analysis.ack_rtt`
- Affects every exchange

Server delay

- ACK → first response byte
- `http.time` for HTTP
- Grows with server load

Client delay

- Response → next request
- Think time or app processing
- Often not a network problem

Time Reference and Timestamps

1

Ctrl+T sets a time reference — times are then measured from that packet.

2

Use a reference per conversation to time one transaction in isolation.

3

Timestamp precision is set by the capture system; you cannot add precision later.

4

Clock offsets between capture points need Time Shift before comparing.

Separate path and server delay

LAB 5

Displayed delta depends on the current filter; capture delta does not.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 5

http.request.uri == "/slow" http.response.code == 200					
No.	Time	Source	Destination	Protocol	Info
23	1.140000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
30	1.230000	192.0.2.10	192.0.2.20	HTTP	GET /slow HTTP/1.1
32	1.990000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
branch-office.pcap Displayed: 3					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Separate path and server delay

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Separate path and server delay

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply `tcp.stream == 1`. Expand TCP in the final handshake ACK and inspect `tcp.analysis.initial_rtt`: 0.040 seconds. Separately measure SYN to SYN/ACK by setting a time reference on the SYN: 0.030 seconds.

Separate path and server delay

STEP 3 OF 6

3

Locate GET /slow and its HTTP response. Set a new time reference on the request. Record the response delta: 0.760 seconds, including 0.010 seconds to the server ACK and 0.750 seconds thereafter.

Separate path and server delay

STEP 4 OF 6

4

Apply `http.request.uri == "/slow" || http.response.code == 200` and compare `frame.time_delta` with `frame.time_delta_displayed` on the filtered list. Explain why a displayed gap can span hidden packets.

Separate path and server delay

STEP 5 OF 6

5

Write a hypothesis in outputs/findings.md: slow response with a synthetic 30 ms SYN-to-SYN/ACK interval and 40 ms complete handshake. Name a second observation point required to distinguish processing from a later path delay.

Separate path and server delay

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```


Separate path and server delay

Test it

The supplied `http.request.uri == "/slow" || http.response.code == 200` expression matches 3 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Time Values and Delay Types

1

You can now: navigate, filter, summarize and time network exchanges.

2

Displayed delta depends on the current filter; capture delta does not.

3

SYN to SYN/ACK gives an initial path-related sample; request to response includes application processing.

4

One-sided timestamps cannot identify exactly which intermediate device delayed or dropped a packet.

Tea Break

15 minutes

TOPIC 06

Trace Statistics and VoIP Overview

Summarize traffic and a voice stream

Key Concepts — Trace Statistics and VoIP Overview

1

Protocol Hierarchy shows captured composition; byte share differs from packet share.

2

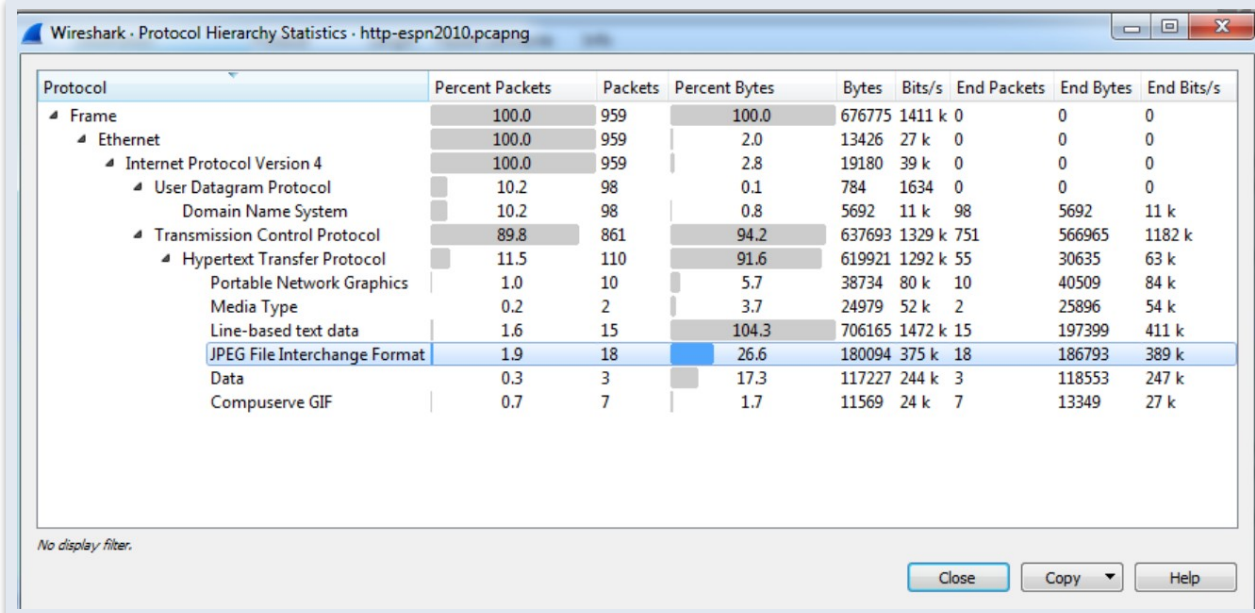
Conversations and Endpoints identify concentration; they do not alone prove malicious activity.

3

SIP signals a call while RTP carries media; jitter, loss and codec interpretation need stream context.

Capture File Properties

- Statistics > Capture File Properties: file, time span, interface and drop counts.
- Check packets dropped before blaming the network for loss.
- Record these facts at the start of every analysis.



Wireshark · Protocol Hierarchy Statistics · http-espn2010.pcapng

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	959	100.0	676775	1411 k	0	0	0
Ethernet	100.0	959	2.0	13426	27 k	0	0	0
Internet Protocol Version 4	100.0	959	2.8	19180	39 k	0	0	0
User Datagram Protocol	10.2	98	0.1	784	1634	0	0	0
Domain Name System	10.2	98	0.8	5692	11 k	98	5692	11 k
Transmission Control Protocol	89.8	861	94.2	637693	1329 k	751	566965	1182 k
Hypertext Transfer Protocol	11.5	110	91.6	619921	1292 k	55	30635	63 k
Portable Network Graphics	1.0	10	5.7	38734	80 k	10	40509	84 k
Media Type	0.2	2	3.7	24979	52 k	2	25896	54 k
Line-based text data	1.6	15	104.3	706165	1472 k	15	197399	411 k
JPEG File Interchange Format	1.9	18	26.6	180094	375 k	18	186793	389 k
Data	0.3	3	17.3	117227	244 k	3	118553	247 k
Compuserve GIF	0.7	7	1.7	11569	24 k	7	13349	27 k

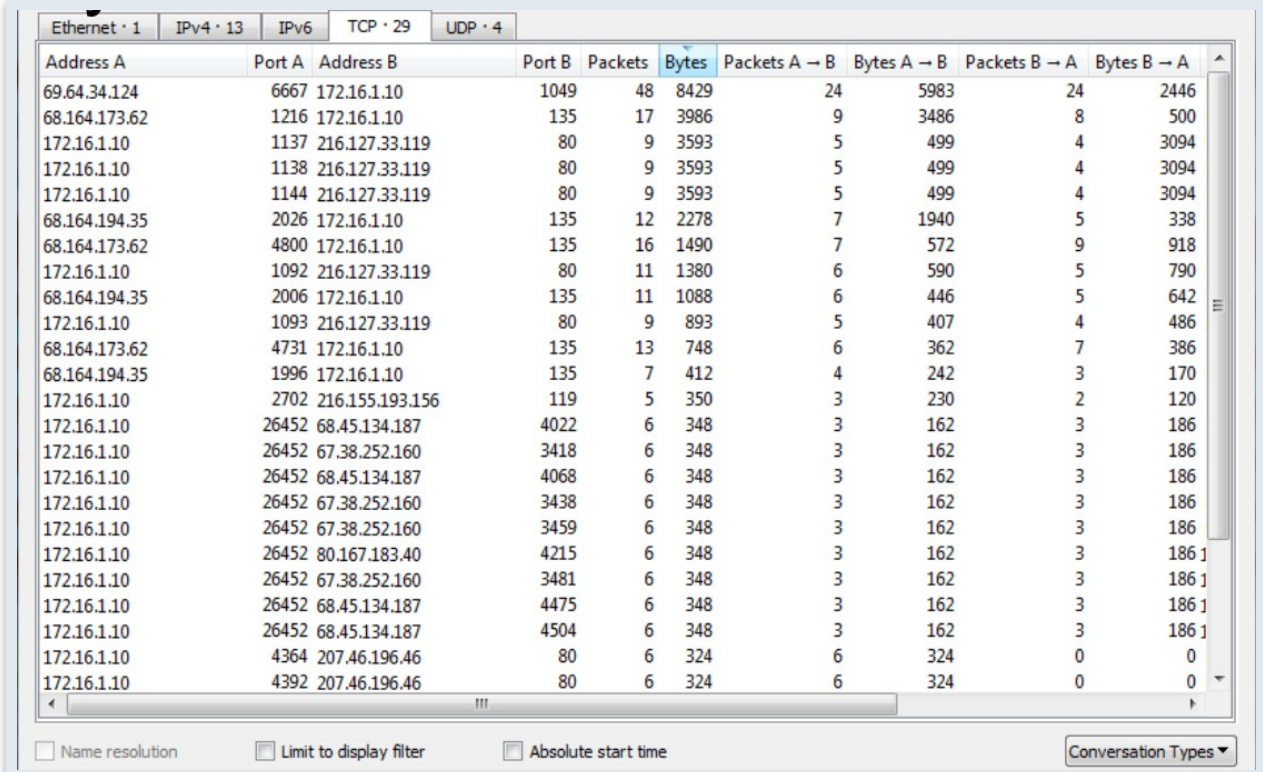
No display filter.

Close Copy Help

Statistics | Protocol Hierarchy shows the protocol mix

Conversations and Endpoints

- Statistics > Conversations lists pairs at Ethernet, IPv4/6, TCP and UDP level.
- Sort by Bytes to find the most active conversation.
- Right-click a row to filter on it.

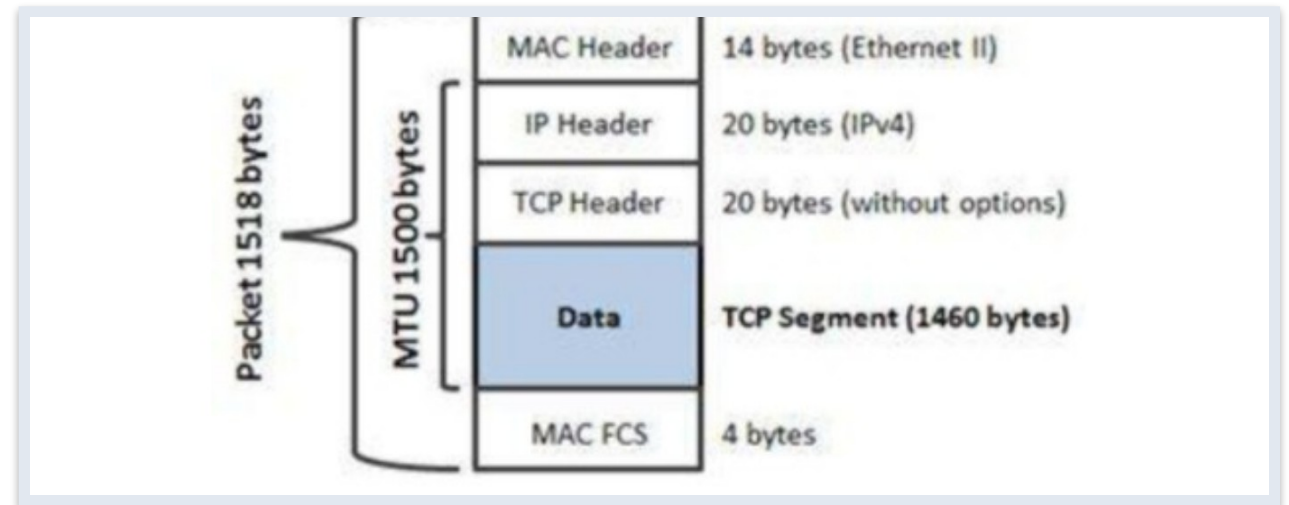


Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A
69.64.34.124	6667	172.16.1.10	1049	48	8429	24	5983	24	2446
68.164.173.62	1216	172.16.1.10	135	17	3986	9	3486	8	500
172.16.1.10	1137	216.127.33.119	80	9	3593	5	499	4	3094
172.16.1.10	1138	216.127.33.119	80	9	3593	5	499	4	3094
172.16.1.10	1144	216.127.33.119	80	9	3593	5	499	4	3094
68.164.194.35	2026	172.16.1.10	135	12	2278	7	1940	5	338
68.164.173.62	4800	172.16.1.10	135	16	1490	7	572	9	918
172.16.1.10	1092	216.127.33.119	80	11	1380	6	590	5	790
68.164.194.35	2006	172.16.1.10	135	11	1088	6	446	5	642
172.16.1.10	1093	216.127.33.119	80	9	893	5	407	4	486
68.164.173.62	4731	172.16.1.10	135	13	748	6	362	7	386
68.164.194.35	1996	172.16.1.10	135	7	412	4	242	3	170
172.16.1.10	2702	216.155.193.156	119	5	350	3	230	2	120
172.16.1.10	26452	68.45.134.187	4022	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3418	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4068	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3438	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3459	6	348	3	162	3	186
172.16.1.10	26452	80.167.183.40	4215	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3481	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4475	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4504	6	348	3	162	3	186
172.16.1.10	4364	207.46.196.46	80	6	324	6	324	0	0
172.16.1.10	4392	207.46.196.46	80	6	324	6	324	0	0

Statistics | Conversations — TCP tab, sorted by bytes

Packet Lengths

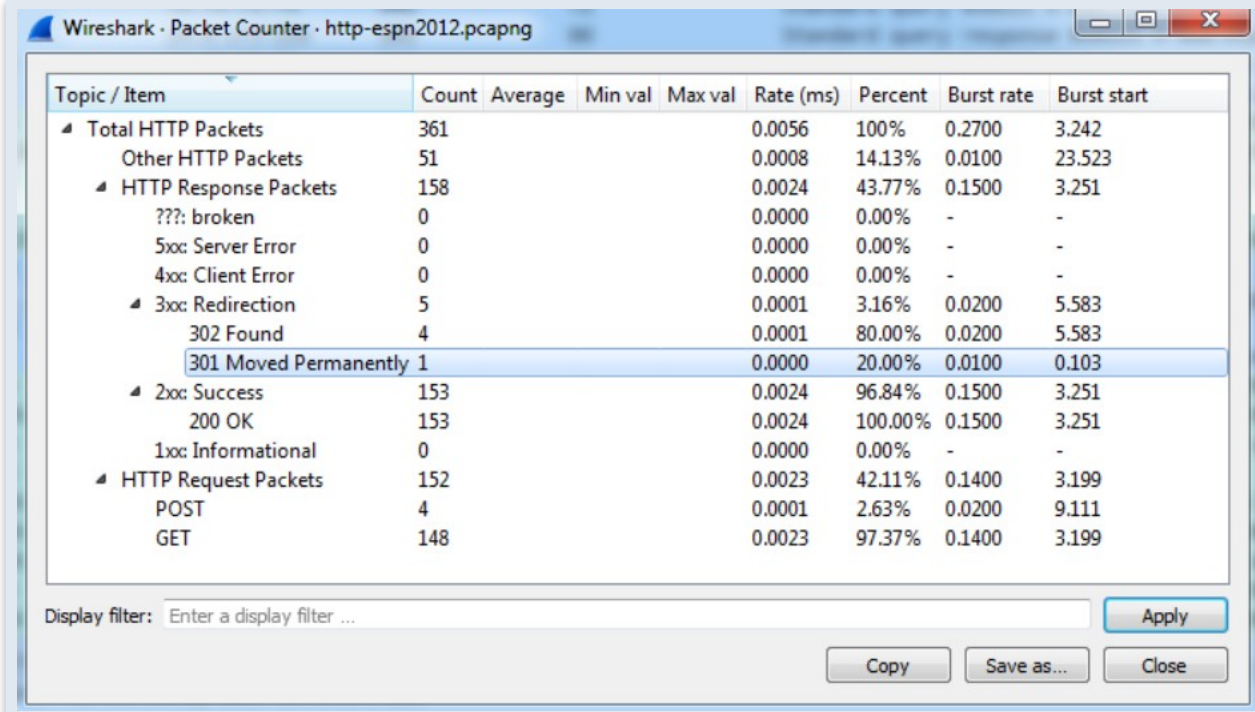
- Statistics > Packet Lengths shows the size distribution.
- Many tiny packets moving bulk data suggest inefficient application behaviour.
- A full-size Ethernet frame is 1518 bytes; TCP payload at MTU 1500 is 1460.



Frame size components

Flow Graph and HTTP Statistics

- Statistics > Flow Graph draws the exchange between hosts as a ladder diagram.
- Statistics > HTTP > Packet Counter summarises requests and response codes.
- Use both to explain sequence and outcome in a report.



The screenshot shows the 'Wireshark · Packet Counter · http-espn2012.pcapng' window. It displays a table of HTTP statistics. The '301 Moved Permanently' entry is highlighted in blue.

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
▲ Total HTTP Packets	361				0.0056	100%	0.2700	3.242
Other HTTP Packets	51				0.0008	14.13%	0.0100	23.523
▲ HTTP Response Packets	158				0.0024	43.77%	0.1500	3.251
??? broken	0				0.0000	0.00%	-	-
5xx: Server Error	0				0.0000	0.00%	-	-
4xx: Client Error	0				0.0000	0.00%	-	-
▲ 3xx: Redirection	5				0.0001	3.16%	0.0200	5.583
302 Found	4				0.0001	80.00%	0.0200	5.583
301 Moved Permanently	1				0.0000	20.00%	0.0100	0.103
▲ 2xx: Success	153				0.0024	96.84%	0.1500	3.251
200 OK	153				0.0024	100.00%	0.1500	3.251
1xx: Informational	0				0.0000	0.00%	-	-
▲ HTTP Request Packets	152				0.0023	42.11%	0.1400	3.199
POST	4				0.0001	2.63%	0.0200	9.111
GET	148				0.0023	97.37%	0.1400	3.199

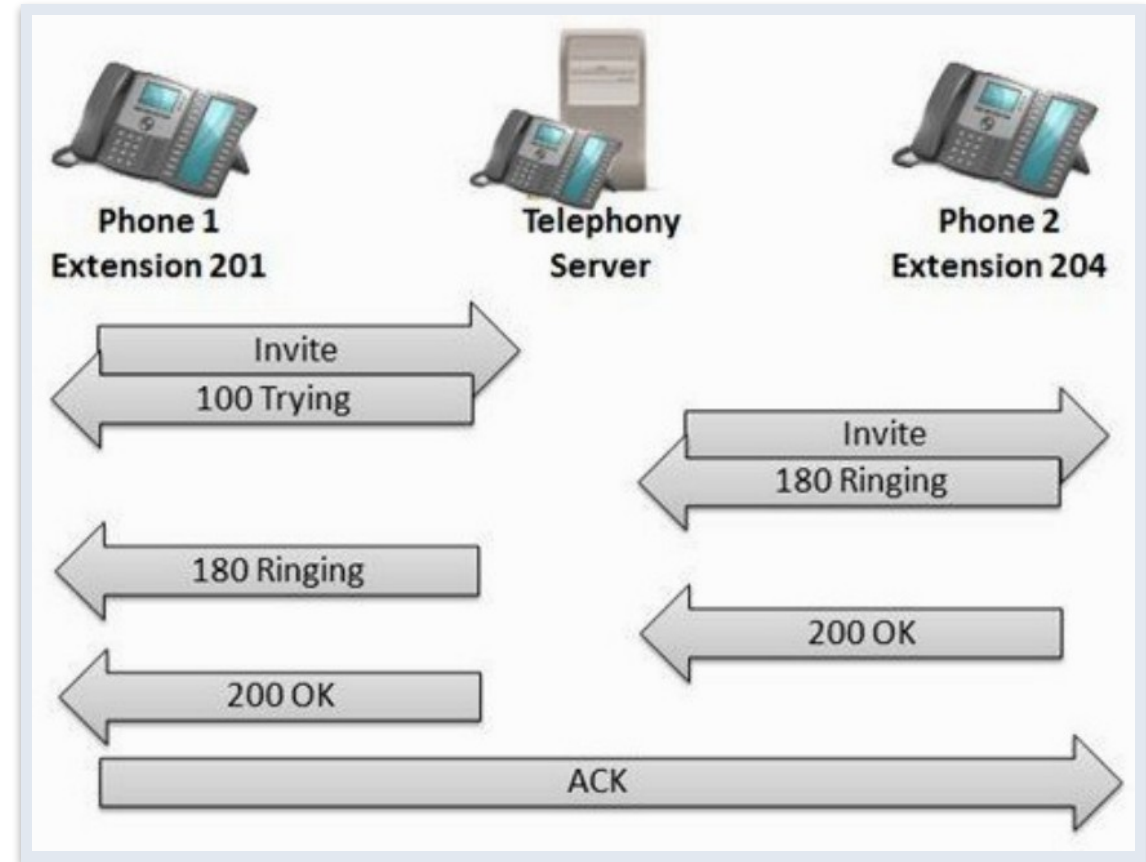
Display filter: Enter a display filter ... Apply

Copy Save as... Close

Statistics | HTTP | Packet Counter

Voice over IP (VoIP)

- SIP signals the call: INVITE, 100 Trying, 180 Ringing, 200 OK, ACK.
- RTP carries the media; gaps in RTP sequence numbers indicate loss.
- Telephony > VoIP Calls, SIP Flows and RTP Streams analyse calls.



SIP call set-up between two phones and a telephony server

Summarize traffic and a voice stream

LAB 6

Protocol Hierarchy shows captured composition; byte share differs from packet share.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 6

sip					
No.	Time	Source	Destination	Protocol	Info
57	3.330000	192.0.2.10	192.0.2.20	SIP	Request: INVITE sip:desk@192.0.2.20
58	3.340000	192.0.2.20	192.0.2.10	SIP	Status: 200 OK (INVITE)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Summarize traffic and a voice stream

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Summarize traffic and a voice stream

STEP 2 OF 6

2

Open data/branch-office.pcap. Open Statistics > Protocol Hierarchy, then Statistics > Conversations. Sort TCP conversations by bytes; save the largest conversation details.

Summarize traffic and a voice stream

STEP 3 OF 6

3

Open Statistics > I/O Graphs. Create an all-traffic graph with a 1 second interval and Bits as the unit. Record peak interval and explain the difference between observed bits and usable application throughput.

Summarize traffic and a voice stream

STEP 4 OF 6

4

Apply sip and open Telephony > VoIP Calls or SIP Flows. Identify the INVITE and 200 OK. This minimal synthetic exchange is signalling evidence, not a complete production call.

Summarize traffic and a voice stream

STEP 5 OF 6

5

Apply `udp.port == 4002`. Use Analyze > Decode As to decode this UDP port as RTP. Open Telephony > RTP > RTP Streams, select the stream and Analyze. Record sequence numbers 100, 101, 103, 104; the missing sequence is 102.

Summarize traffic and a voice stream

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Summarize traffic and a voice stream

Test it

The supplied sip expression matches 2 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

Recap — Trace Statistics and VoIP Overview

1

You can now: navigate, filter, summarize and time network exchanges.

2

Protocol Hierarchy shows captured composition; byte share differs from packet share.

3

Conversations and Endpoints identify concentration; they do not alone prove malicious activity.

4

SIP signals a call while RTP carries media; jitter, loss and codec interpretation need stream context.

Lunch Break

1 hour

TOPIC 07

Display Filters

Build a filter evidence matrix

Key Concepts — Display Filters

1

Parentheses make mixed and/or expressions explicit.

2

Field existence and Boolean equality differ: `tcp.flags.syn == 1` tests the bit.

3

Since Wireshark 3.6, `!=` uses all-not-equal semantics; historical slides describing any-not-equal are obsolete.

Display Filter Operators

- Comparison: == != > < >= <= (or eq ne gt lt ge le).
- Logical: && (and), || (or), ! (not); parentheses make intent explicit.
- Strings in double quotes; contains and matches search payload text.

Shortcut	C-Like	Description	Example
eq	==	Equal	Ip.src == 10.1.1.5
ne	!=	Not equal	Ip.src != 10.1.1.5
gt	>	Greater than	Frame.len > 64
lt	<	Less Than	Frame.len < 1518
ge	>=	Greater then or equal to	Frame.len ge 0x100
le	<=	Less then or equal to	Frame.len <= 0x20

Comparison operators with C-like and English forms

The != Trap (Wireshark 3.6+)

`ip.addr != 10.2.4.1`

- Means ALL ip.addr fields are not 10.2.4.1
- Excludes every packet to OR from 10.2.4.1
- This is the current behaviour

`ip.addr ~= 10.2.4.1`

- "Any not equal" — the old meaning of !=
- Keeps packets where either address differs
- Prefer `!(ip.addr == 10.2.4.1)` for clarity

Filter Building Techniques

1 Right-click a field > Apply as Filter / Prepare as Filter.

2 Start typing — autocomplete lists valid field names.

3 Save frequent filters as filter buttons in your profile.

4 Test field existence (`tcp.analysis.flags`) vs a value (`tcp.flags.syn == 1`).

Filters You Will Use All Week

1

arp || icmp

Local resolution and reachability

2

dns.flags.rcode != 0

DNS errors

3

tcp.flags.syn == 1 && tcp.flags.ack == 0

Connection attempts

4

tcp.analysis.flags

All TCP expert warnings

5

http.response.code >= 400

Client and server errors

6

frame.time_delta_displayed > 1

Gaps over 1 s in the view

Build a filter evidence matrix

LAB 7

Parentheses make mixed and/or expressions explicit.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 7

dns.flags.rcode == 3					
No.	Time	Source	Destination	Protocol	Info
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Build a filter evidence matrix

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Build a filter evidence matrix

STEP 2 OF 6



2

Open data/branch-office.pcap. Apply `dns.flags.rcode == 3` and record the returned response frame. Clear the filter and apply `tcp.flags.syn == 1`.

Build a filter evidence matrix

STEP 3 OF 6

3

Compare `(ip.src == 192.0.2.10 && udp.port == 53) || tcp.port == 80` with `ip.src == 192.0.2.10 && (udp.port == 53 || tcp.port == 80)`. Record both counts and the direction excluded by the second expression.

Build a filter evidence matrix

STEP 4 OF 6

4

Apply `!(ip.addr == 192.0.2.53)`. Verify that DNS packets disappear; compare `ip.addr != 192.0.2.53` on Wireshark 4.6 or later. Record the actual version and equality behaviour.

Build a filter evidence matrix

STEP 5 OF 6

5

Save five useful filters in `outputs/filters.txt`. For each, state the question, filter and matching frame numbers; compile them with `scripts/verify.py`.

Build a filter evidence matrix

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Build a filter evidence matrix

Test it

The supplied `dns.flags.rcode == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Display Filters

1

You can now: navigate, filter, summarize and time network exchanges.

2

Parentheses make mixed and/or expressions explicit.

3

Field existence and Boolean equality differ: `tcp.flags.syn == 1` tests the bit.

4

Since Wireshark 3.6, `!=` uses all-not-equal semantics; historical slides describing any-not-equal are obsolete.

TOPIC 08

TCP/IP Communications and Resolution

Reconstruct an application dependency chain

Key Concepts — TCP/IP Communications and Resolution

1

ARP resolves a local next-hop MAC; DNS resolves a name to an address.

2

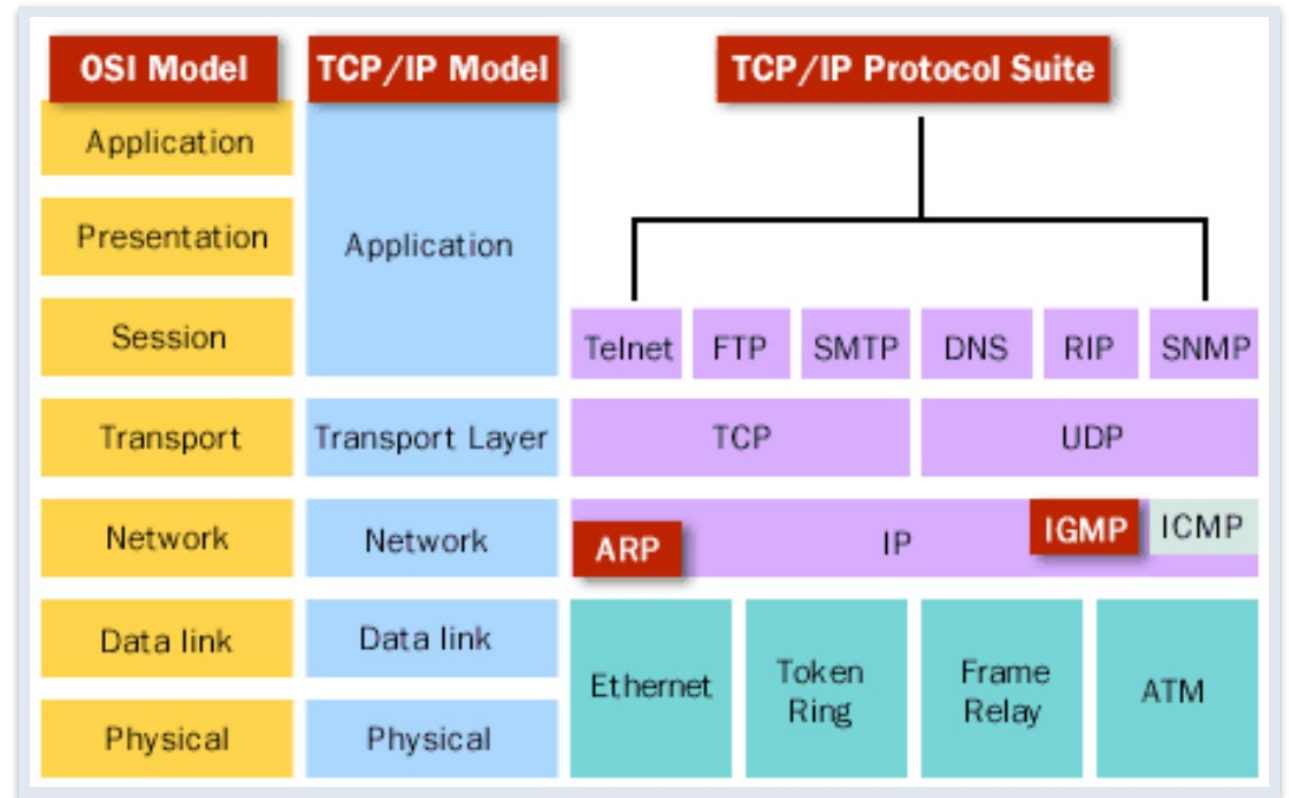
The destination IP stays end-to-end across routing while link-layer addresses change per hop.

3

Resolution, connection establishment and application exchange form a dependency chain.

The TCP/IP Protocol Suite

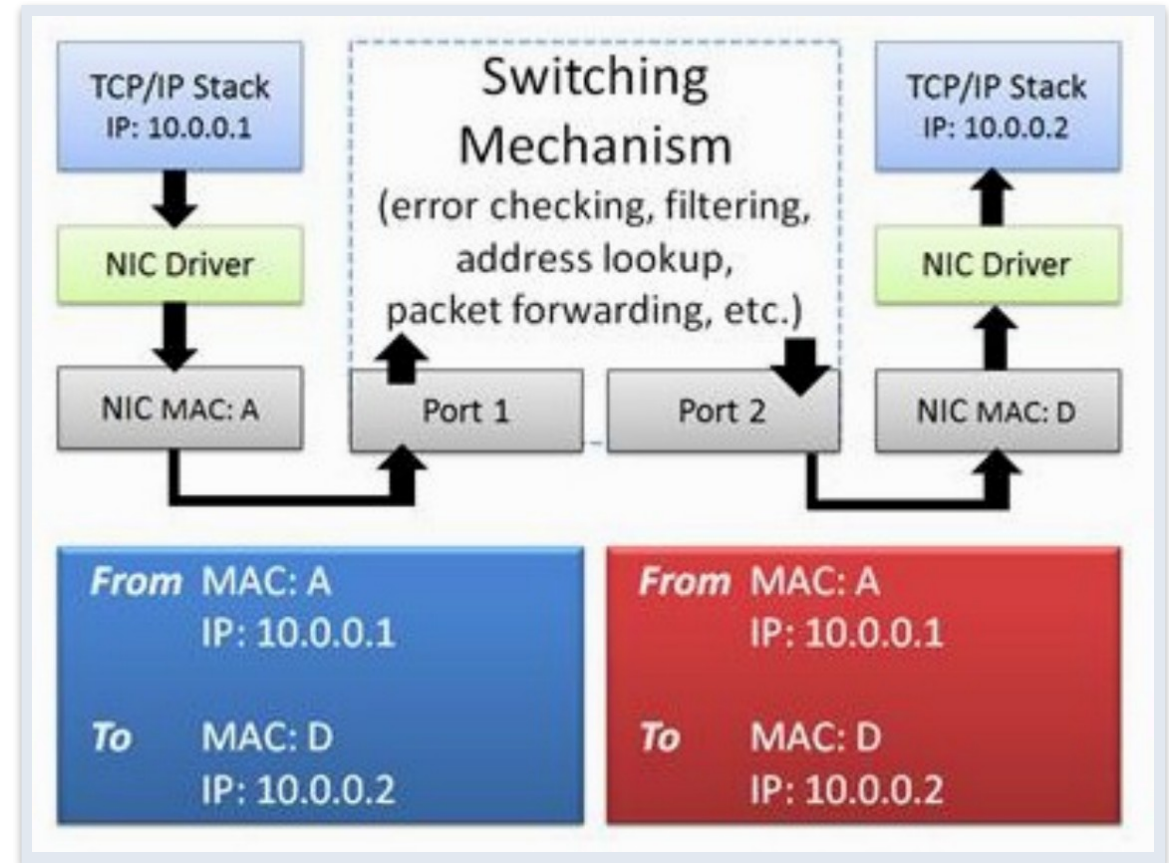
- Application: HTTP, DNS, SMTP, SNMP ...
- Transport: TCP (reliable) and UDP (best-effort).
- Network: IP, ICMP, ARP sits between network and link.



Where common protocols sit

Switching Overview

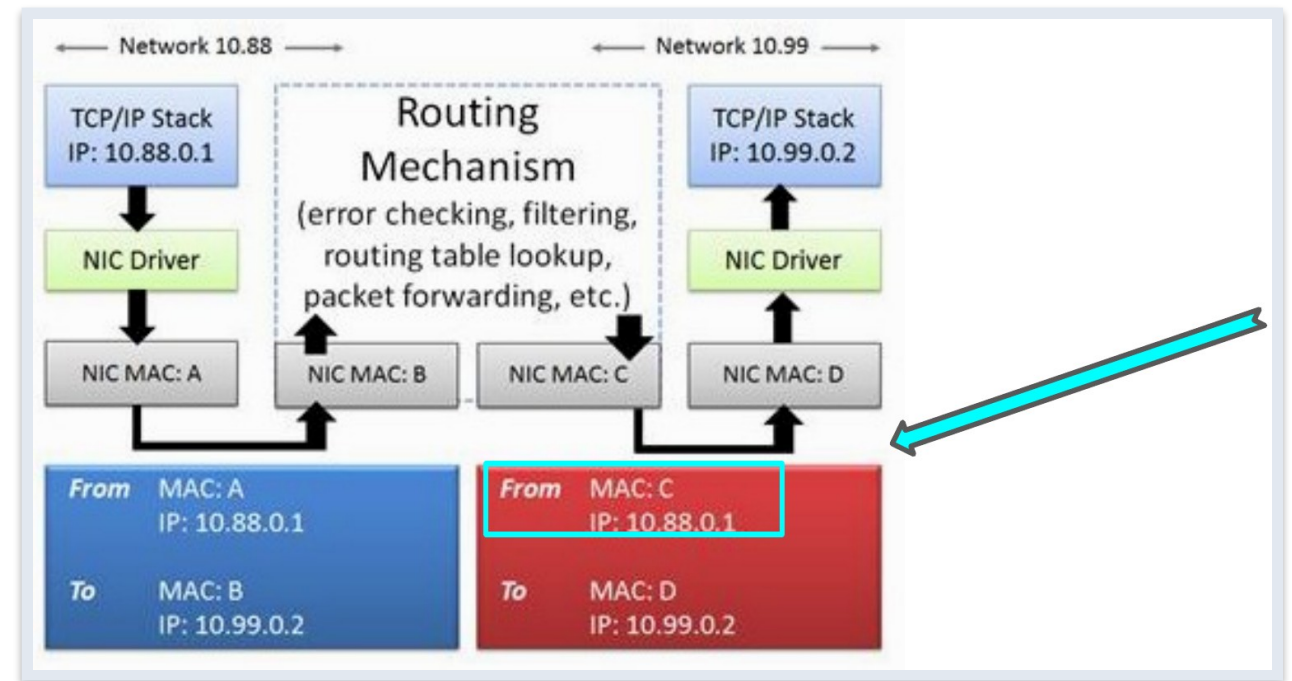
- Switches learn MAC addresses and forward frames within a broadcast domain.
- Source and destination MAC stay the same across a switch.
- ARP resolves the next-hop MAC before the first frame is sent.



A frame crossing a switch keeps its MAC addresses

Routing Overview

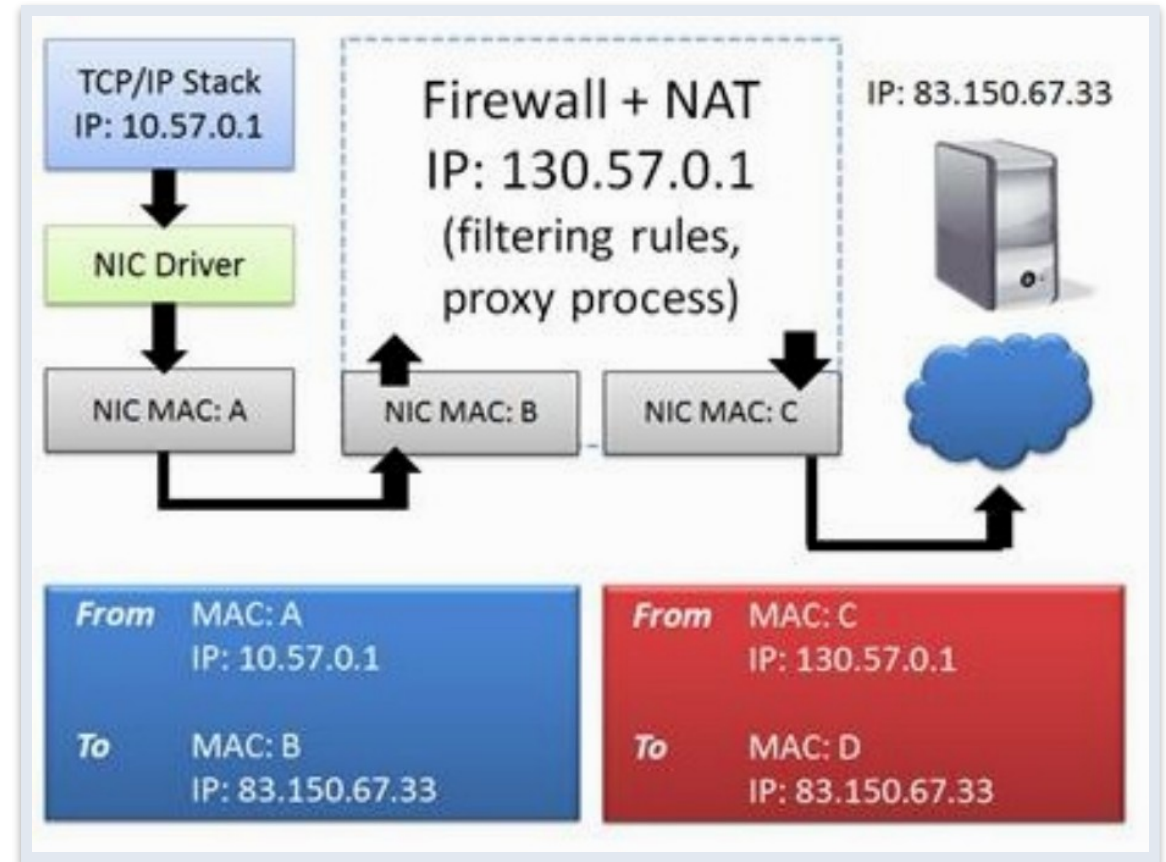
- Routers forward on destination IP and rewrite the link-layer addresses.
- TTL decreases by one at every router hop.
- The IP addresses are unchanged across routers (without NAT).



MAC addresses change hop-by-hop; IP addresses stay end-to-end

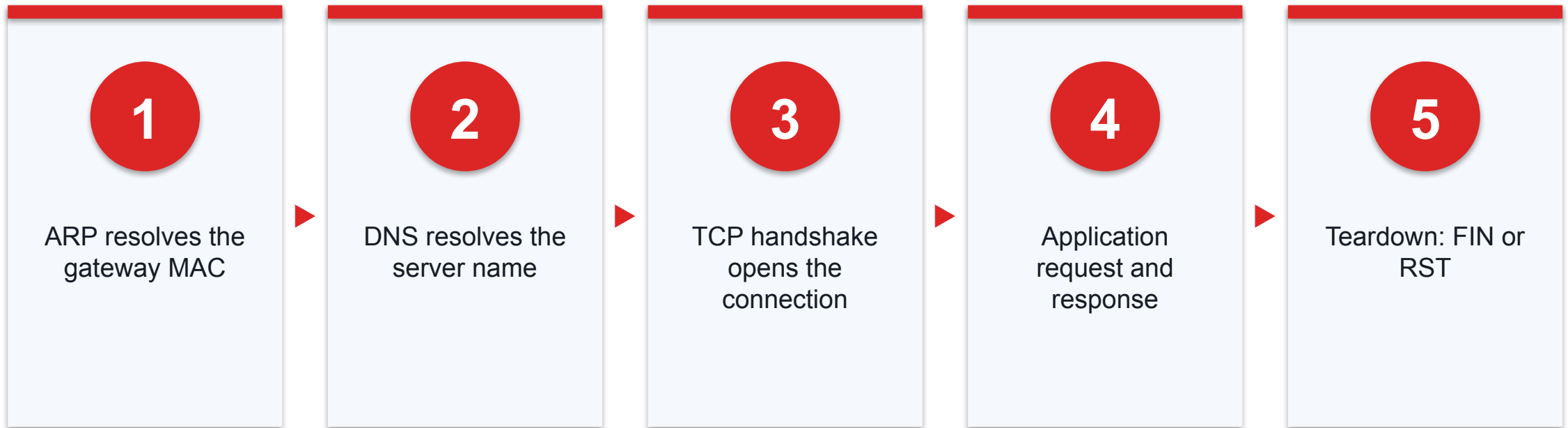
Firewalls and NAT

- NAT/PAT rewrites addresses and ports — capture on both sides to correlate.
- Firewalls may drop silently or reply with RST / ICMP unreachable.
- A proxy creates two separate TCP connections.



Address translation changes what each capture point sees

The Dependency Chain



Reconstruct an application dependency chain

LAB 8

ARP resolves a local next-hop MAC; DNS resolves a name to an address.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 8

dns.qry.name == "portal.example.test"					
No.	Time	Source	Destination	Protocol	Info
3	0.020000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0065 A portal.example.test
4	0.040000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0065 A portal.example.t...
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Reconstruct an application dependency chain

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Reconstruct an application dependency chain

STEP 2 OF 6

2

Open data/branch-office.pcap. Locate the ARP exchange and DNS transaction 101 for portal.example.test. Record the server address 192.0.2.20.

Reconstruct an application dependency chain

STEP 3 OF 6

3

Apply `tcp.stream == 0` and open Statistics > Flow Graph with displayed packets selected. Identify SYN, SYN/ACK, ACK, request and response.

Reconstruct an application dependency chain

STEP 4 OF 6

4

Fill assets/dependency-map.md with the ARP, DNS, TCP and HTTP frame ranges. Distinguish Ethernet next hop from IP destination.

Reconstruct an application dependency chain

STEP 5 OF 6



5

Contrast a same-subnet server with a routed server using `assets/topology.md`: a routed destination needs the gateway MAC rather than ARP for the remote host. Record what this trace can and cannot demonstrate.

Reconstruct an application dependency chain

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Reconstruct an application dependency chain

Test it

The supplied `dns.qry.name == "portal.example.test"` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TCP/IP Communications and Resolution

1

You can now: navigate, filter, summarize and time network exchanges.

2

ARP resolves a local next-hop MAC; DNS resolves a name to an address.

3

The destination IP stays end-to-end across routing while link-layer addresses change per hop.

4

Resolution, connection establishment and application exchange form a dependency chain.

Tea Break

15 minutes

TOPIC 09

DNS Traffic Analysis

Diagnose DNS failure and delay

Key Concepts — DNS Traffic Analysis

1

Transaction ID plus addresses and ports pair a query with its response.

2

NXDOMAIN reports that a name does not exist; it differs from silence or timeout.

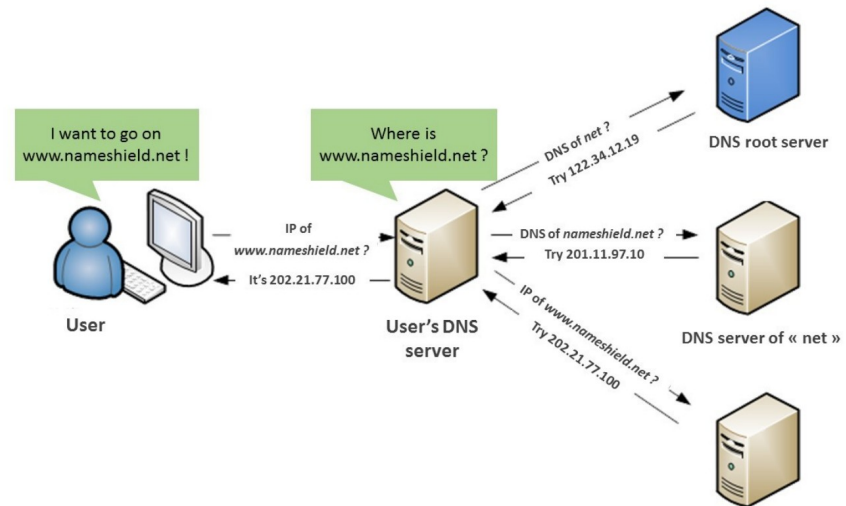
3

DNS response time is a measured exchange, influenced by resolver and path behaviour.

How DNS Works

- The client asks its resolver; the resolver walks the hierarchy or answers from cache.
- Queries normally use UDP 53; large answers and zone transfers use TCP.
- Transaction ID pairs each response with its query.

Domain Name System (DNS) provides host name-to-IP address resolution services.



Name resolution through a local resolver

Reading DNS Evidence

Success

- rcode 0 (NoError)
- Answer records present
- Check TTL and address

Failure

- rcode 3 = NXDOMAIN
- rcode 2 = SERVFAIL
- Name or zone problem

Silence

- Query with no response
- Retries after timeout
- Path, firewall or resolver

Useful DNS Filters

1 `dns.flags.response == 0` — queries only.

2 `dns.flags.rcode == 3` — name does not exist.

3 `dns.time > 0.5` — responses slower than 500 ms.

4 `dns.qry.name contains "example"` — one domain family.

Diagnose DNS failure and delay

LAB 9

Transaction ID plus addresses and ports pair a query with its response.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 9

dns.flags.rcode == 3					
No.	Time	Source	Destination	Protocol	Info
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Diagnose DNS failure and delay

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Diagnose DNS failure and delay

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply dns and add dns.time as a column from a response packet. Pair IDs 101, 102 and 103.

Diagnose DNS failure and delay

STEP 3 OF 6

3

Apply `dns.flags.rcode == 3`. Expand DNS flags and name. Record missing.example.test and explain NXDOMAIN.

Diagnose DNS failure and delay

STEP 4 OF 6

4

Apply `dns.id == 103`. Measure the query-response interval: 0.800 seconds. Compare with the two 0.020 second transactions.

Diagnose DNS failure and delay

STEP 5 OF 6



5

Complete assets/dns-observations.csv with name, transaction ID, rcode, elapsed seconds and next investigation. Do not label this measured delay as a DNS timeout.

Diagnose DNS failure and delay

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Diagnose DNS failure and delay

Test it

The supplied `dns.flags.rcode == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — DNS Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

Transaction ID plus addresses and ports pair a query with its response.

3

NXDOMAIN reports that a name does not exist; it differs from silence or timeout.

4

DNS response time is a measured exchange, influenced by resolver and path behaviour.

End of Day 2

See you tomorrow

DAY 3

IP and transport diagnostics

Recap of the previous day, then today's topics and labs

TOPIC 10

ARP Traffic Analysis

Verify link-local resolution

Key Concepts — ARP Traffic Analysis

1

ARP requests are link-local broadcasts; replies usually return to the requester.

2

A repeated unresolved request suggests a local resolution problem, but a single trace may miss the reply.

3

MAC changes require corroboration before claiming duplicate IP or spoofing.

ARP in Practice

Request

- Broadcast to ff:ff:ff:ff:ff:ff
- "Who has 192.0.2.20? Tell 192.0.2.10"
- arp.opcode == 1

Reply

- Unicast back to the requester
- "192.0.2.20 is at 02:00:..."
- arp.opcode == 2

Gratuitous

- Sender announces its own IP
- Used after failover / IP change
- arp.isgratuitous

ARP Problems to Recognise

Symptoms

- Repeated requests with no reply
- Duplicate IP address detected warnings
- One IP mapped to two MAC addresses
- Excessive ARP broadcasts

Possible causes

- Host down or wrong subnet
- Two hosts configured with the same IP
- ARP spoofing — corroborate first
- Scanning or misconfigured devices

Verify link-local resolution

LAB 10

ARP requests are link-local broadcasts; replies usually return to the requester.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 10

arp.opcode == 2					
No.	Time	Source	Destination	Protocol	Info
2	0.010000	02:00:00:00:00:20	02:00:00:00:00:10	ARP	192.0.2.20 is at 02:00:00:00:00:20
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Verify link-local resolution

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Verify link-local resolution

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply arp. Expand the request and reply, then record sender IP, sender MAC, target IP and opcode.

Verify link-local resolution

STEP 3 OF 6

3

Apply `arp.opcode == 2`. Confirm the advertised server mapping is 192.0.2.20 to 02:00:00:00:00:20.

Verify link-local resolution

STEP 4 OF 6

4

Use the packet bytes to verify the opcode field is 2 in the reply. Add the mapping to `assets/arp-table.csv`.

Verify link-local resolution

STEP 5 OF 6

5

Compare assets/arp-hypotheses.md with the evidence. State that this capture shows one successful exchange, without inventing repeated failures or conflicting MACs.

Verify link-local resolution

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Verify link-local resolution

Test it

The supplied `arp.opcode == 2` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — ARP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

ARP requests are link-local broadcasts; replies usually return to the requester.

3

A repeated unresolved request suggests a local resolution problem, but a single trace may miss the reply.

4

MAC changes require corroboration before claiming duplicate IP or spoofing.

Tea Break

15 minutes

TOPIC 11

IPv4 Traffic Analysis

Classify IPv4 scope and headers

Key Concepts — IPv4 Traffic Analysis

1

TTL limits forwarding hops; it is not a latency measurement.

2

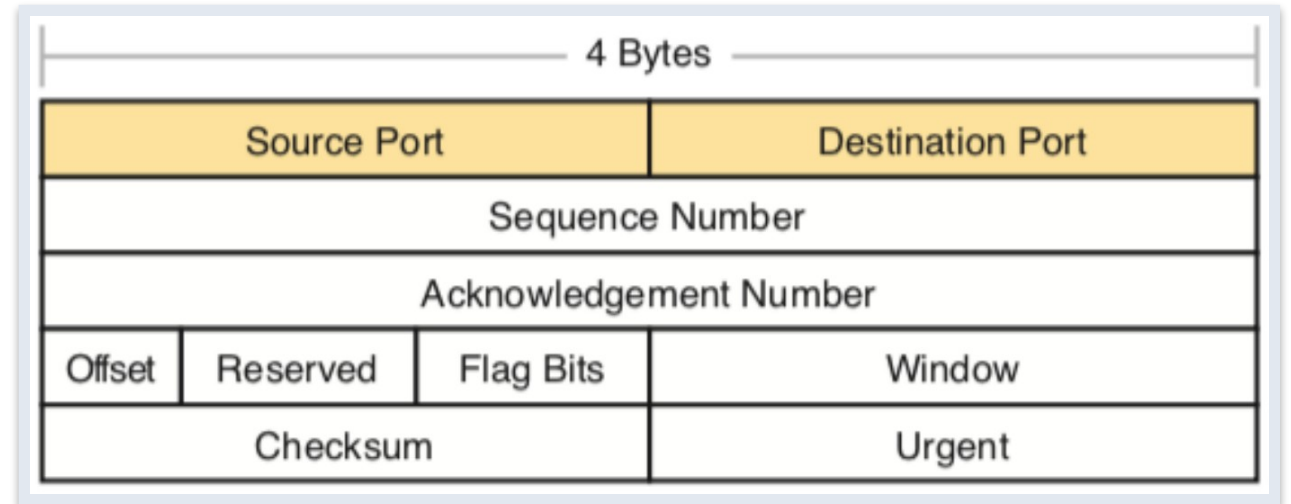
Broadcast and multicast use different addressing scopes and delivery rules.

3

Fragmentation fields describe packet handling; missing fragments can reflect capture limitations.

IPv4 Header Essentials

- TTL — hops remaining; not a measure of time.
- Protocol — 1 ICMP, 6 TCP, 17 UDP.
- Flags/Fragment offset — fragmentation handling.
- DSCP — quality-of-service marking.



Header layout (TCP header shown; IPv4 uses the same 32-bit rows)

Address Scope

Unicast

- One sender to one receiver
- Most application traffic
- `ip.dst == host`

Broadcast

- All hosts on the subnet
- `255.255.255.255` or directed
- Not forwarded by routers

Multicast

- `224.0.0.0/4` group addresses
- `224.0.0.1` = all hosts
- Joined with IGMP

IPv4 Filters and Checks

1 `ip.ttl < 10` — packets close to expiring.

2 `ip.flags.mf == 1 || ip.frag_offset > 0` — fragments.

3 `ip.dst == 224.0.0.0/4` — all multicast.

4 Missing fragments may be a capture limitation — check drops first.

Classify IPv4 scope and headers

LAB 11

TTL limits forwarding hops; it is not a latency measurement.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 11

ip.dst == 224.0.0.1					
No.	Time	Source	Destination	Protocol	Info
17	1.030000	192.0.2.10	224.0.0.1	UDP	55001 → 5000 Len=13
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Classify IPv4 scope and headers

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Classify IPv4 scope and headers

STEP 2 OF 6

2

Open data/branch-office.pcap. Select an ICMP echo request. Expand IPv4 and record version, IHL, total length, TTL, protocol, flags and fragment offset.

Classify IPv4 scope and headers

STEP 3 OF 6

3

Apply `ip.dst == 224.0.0.1` and compare Ethernet destination `01:00:5e:00:00:01` with IPv4 multicast destination. Record TTL 1.

Classify IPv4 scope and headers

STEP 4 OF 6

4

Apply `ip.flags.mf == 1 || ip.frag_offset > 0`. Confirm no fragmented packets in this fixture; absence here does not demonstrate a universal MTU.

Classify IPv4 scope and headers

STEP 5 OF 6

5

Complete assets/ip-header.csv. Explain why ARP broadcast is not an IPv4 broadcast packet, and why a capture at one point cannot estimate hop count from TTL without knowing initial TTL.

Classify IPv4 scope and headers

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Classify IPv4 scope and headers

Test it

The supplied `ip.dst == 224.0.0.1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — IPv4 Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

TTL limits forwarding hops; it is not a latency measurement.

3

Broadcast and multicast use different addressing scopes and delivery rules.

4

Fragmentation fields describe packet handling; missing fragments can reflect capture limitations.

Lunch Break

1 hour

TOPIC 12

ICMP Traffic Analysis

Interpret echo and unreachable messages

Key Concepts — ICMP Traffic Analysis

1

Echo requests/replies show an IP exchange; they do not guarantee an application works.

2

Destination unreachable messages quote the offending datagram.

3

ICMP may be filtered or rate-limited, so absence of a reply needs cautious interpretation.

ICMP Messages to Know

Echo

- Type 8 request, Type 0 reply
- ping / reachability
- Match on identifier + sequence

Unreachable

- Type 3
- Code 3 = port unreachable
- Code 1 = host unreachable

Time exceeded

- Type 11
- TTL expired in transit
- How traceroute works

Interpreting ICMP Evidence

1

An unreachable message quotes the original IP header — read it to find the failed flow.

2

No echo reply can mean filtering, not a down host.

3

Rate-limited ICMP can hide real loss.

4

`icmp.type == 3 && icmp.code == 3` finds closed UDP ports.

Interpret echo and unreachable messages

LAB 12

Echo requests/replies show an IP exchange; they do not guarantee an application works.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 12

icmp.type == 3 && icmp.code == 3					
No.	Time	Source	Destination	Protocol	Info
16	1.020000	192.0.2.20	192.0.2.10	ICMP	Destination unreachable (Port unreachable)
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Interpret echo and unreachable messages

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Interpret echo and unreachable messages

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply `icmp.type == 8 || icmp.type == 0`. Pair echo sequences 0, 1 and 2 using identifier 7. Measure 0.030 seconds per pair.

Interpret echo and unreachable messages

STEP 3 OF 6

3

Apply `icmp.type == 3 && icmp.code == 3`. Expand the quoted IPv4 and UDP headers. Record destination UDP port 9999.

Interpret echo and unreachable messages

STEP 4 OF 6

4

Apply `udp.dstport == 9999` to locate the initiating datagram. Record the difference between a UDP service refusal and an echo reply.

Interpret echo and unreachable messages

STEP 5 OF 6

5

Complete assets/icmp-events.csv with type, code, quoted protocol, port and conclusion. State which evidence supports a service refusal.

Interpret echo and unreachable messages

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Interpret echo and unreachable messages

Test it

The supplied `icmp.type == 3 && icmp.code == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — ICMP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

Echo requests/replies show an IP exchange; they do not guarantee an application works.

3

Destination unreachable messages quote the offending datagram.

4

ICMP may be filtered or rate-limited, so absence of a reply needs cautious interpretation.

TOPIC 13

UDP Traffic Analysis

Follow datagrams and service refusal

Key Concepts — UDP Traffic Analysis

1

UDP has no transport handshake or retransmission; the application may provide reliability.

2

A port number is a decoding hint; payload and context identify the application.

3

Following a UDP stream collects datagrams and does not invent missing data.

TCP vs UDP

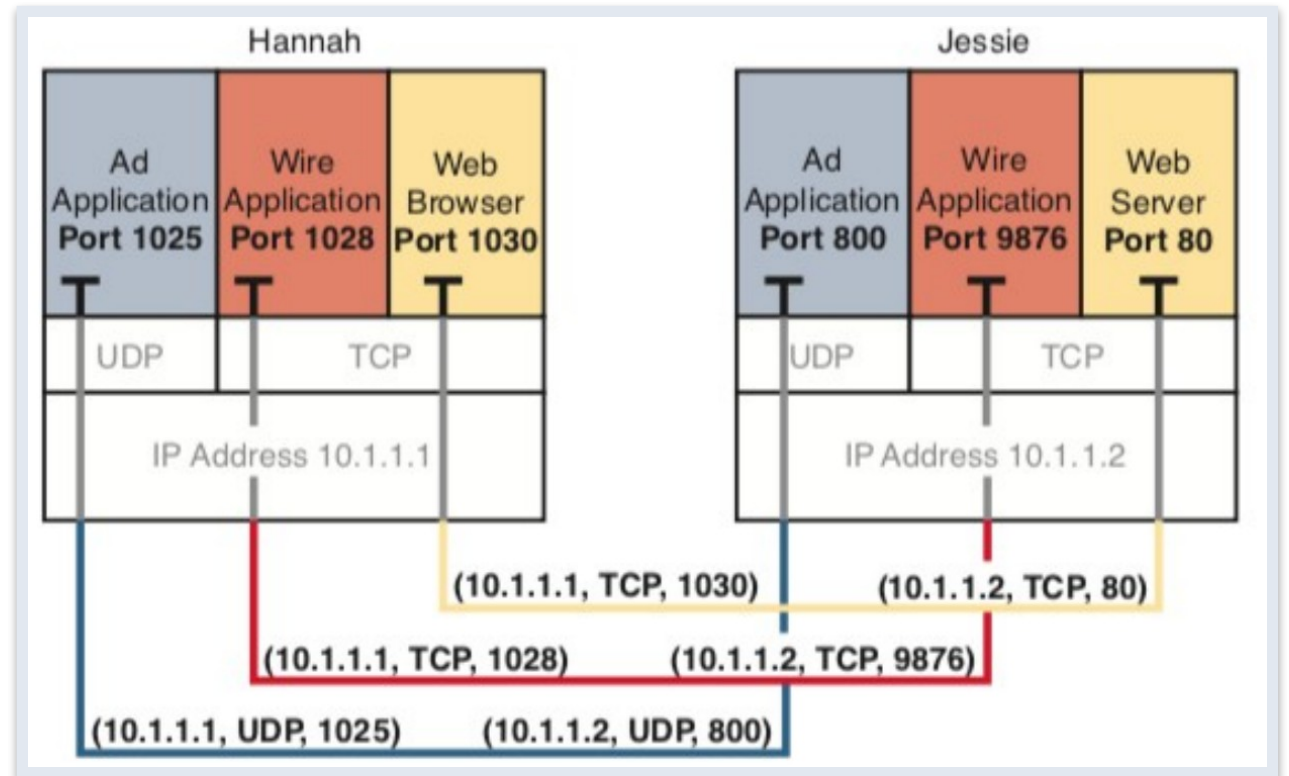
- UDP: connectionless, 8-byte header, no retransmission or ordering.
- TCP: connection-oriented, reliable, ordered, flow-controlled.
- Choose the analysis by the transport: UDP problems show up at the application.

	TCP	UDP																							
Acronym	Transmission Control Protocol	User Datagram Protocol																							
Reliability	High	Best Effort																							
Header Size	4 Bytes <table><tr><td colspan="2">Source Port</td><td colspan="2">Destination Port</td></tr><tr><td colspan="4">Sequence Number</td></tr><tr><td colspan="4">Acknowledgement Number</td></tr><tr><td>Offset</td><td>Reserved</td><td>Flag Bits</td><td>Window</td></tr><tr><td colspan="2">Checksum</td><td colspan="2">Urgent</td></tr></table> Figure 5-1 TCP Header Fields 4x5=20 Byte 4 Bytes <table><tr><td>Source Port</td><td>Destination Port</td></tr><tr><td>Length</td><td>Checksum</td></tr></table> Figure 5-10 UDP Header 4x2=8 Byte	Source Port		Destination Port		Sequence Number				Acknowledgement Number				Offset	Reserved	Flag Bits	Window	Checksum		Urgent		Source Port	Destination Port	Length	Checksum
Source Port		Destination Port																							
Sequence Number																									
Acknowledgement Number																									
Offset	Reserved	Flag Bits	Window																						
Checksum		Urgent																							
Source Port	Destination Port																								
Length	Checksum																								

Header and feature comparison

Ports and Sockets

- A socket is IP address + port.
- A conversation is the pair of sockets — the 5-tuple with the protocol.
- Ephemeral client ports change per connection.



Socket pairs identify each conversation

Analysing UDP

1

Follow > UDP Stream reassembles a datagram conversation.

2

A UDP request answered by ICMP port unreachable = closed port.

3

Missing responses need application-level timing to interpret.

4

Decode As lets you dissect traffic on non-standard ports.

Follow datagrams and service refusal

LAB 13

UDP has no transport handshake or retransmission; the application may provide reliability.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 13

udp.dstport == 9999					
No.	Time	Source	Destination	Protocol	Info
15	1.010000	192.0.2.10	192.0.2.20	UDP	55000 → 9999 Len=7
16	1.020000	192.0.2.20	192.0.2.10	ICMP	Destination unreachable (Port unreachable)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Follow datagrams and service refusal

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Follow datagrams and service refusal

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply `udp.dstport == 9999`. One frame is the original datagram and one is the UDP header quoted inside ICMP; inspect the outer protocol to distinguish them.

Follow datagrams and service refusal

STEP 3 OF 6

3

Select the original datagram and use Analyze > Follow > UDP Stream. Confirm the payload LAB-UDP. Save a text view to outputs/udp-stream.txt.

Follow datagrams and service refusal

STEP 4 OF 6

4

Apply `udp.port == 4002` and decode RTP using Analyze > Decode As. Compare datagram framing with the earlier TCP stream.

Follow datagrams and service refusal

STEP 5 OF 6

5

Record why no UDP transport ACK exists in the trace. Use the ICMP refusal to explain why an application can fail even though a datagram was transmitted.

Follow datagrams and service refusal

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Follow datagrams and service refusal

Test it

The supplied `udp.dstport == 9999` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — UDP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

UDP has no transport handshake or retransmission; the application may provide reliability.

3

A port number is a decoding hint; payload and context identify the application.

4

Following a UDP stream collects datagrams and does not invent missing data.

Tea Break

15 minutes

TOPIC 14

TCP Protocol Analysis

Investigate retransmission and zero window

Key Concepts — TCP Protocol Analysis

1

Sequence numbers count bytes; ACK numbers indicate the next expected byte.

2

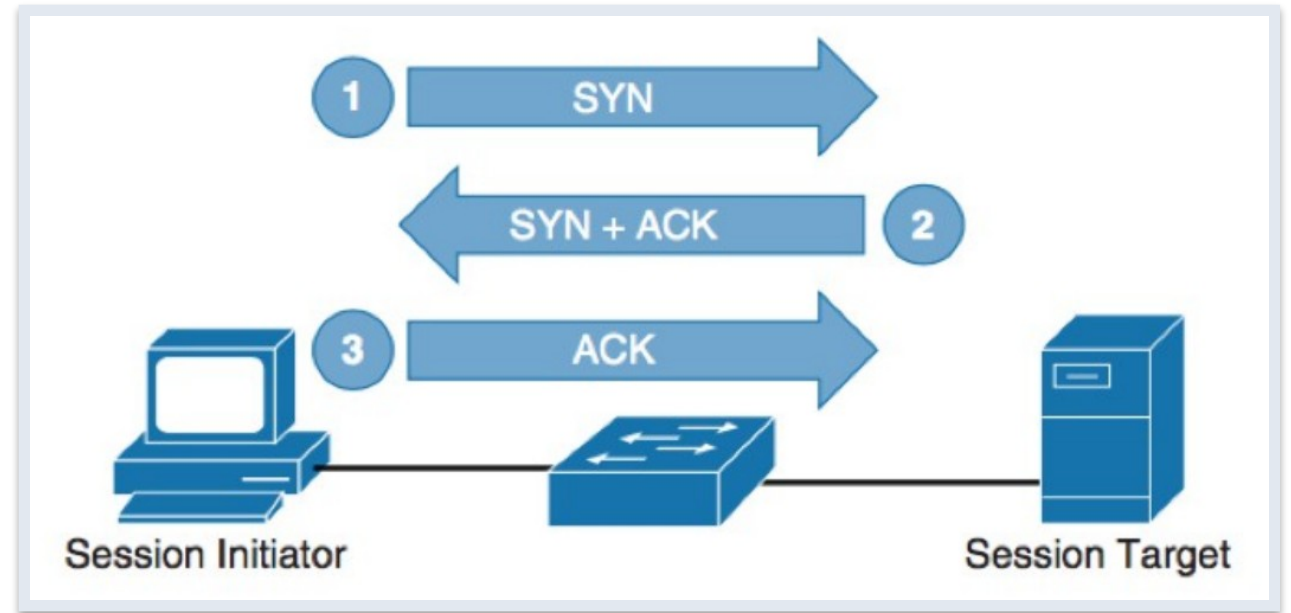
Retransmission and duplicate ACK labels are heuristics sensitive to capture placement and completeness.

3

Zero window indicates receive-side flow control; it is distinct from network congestion.

The Three-Way Handshake

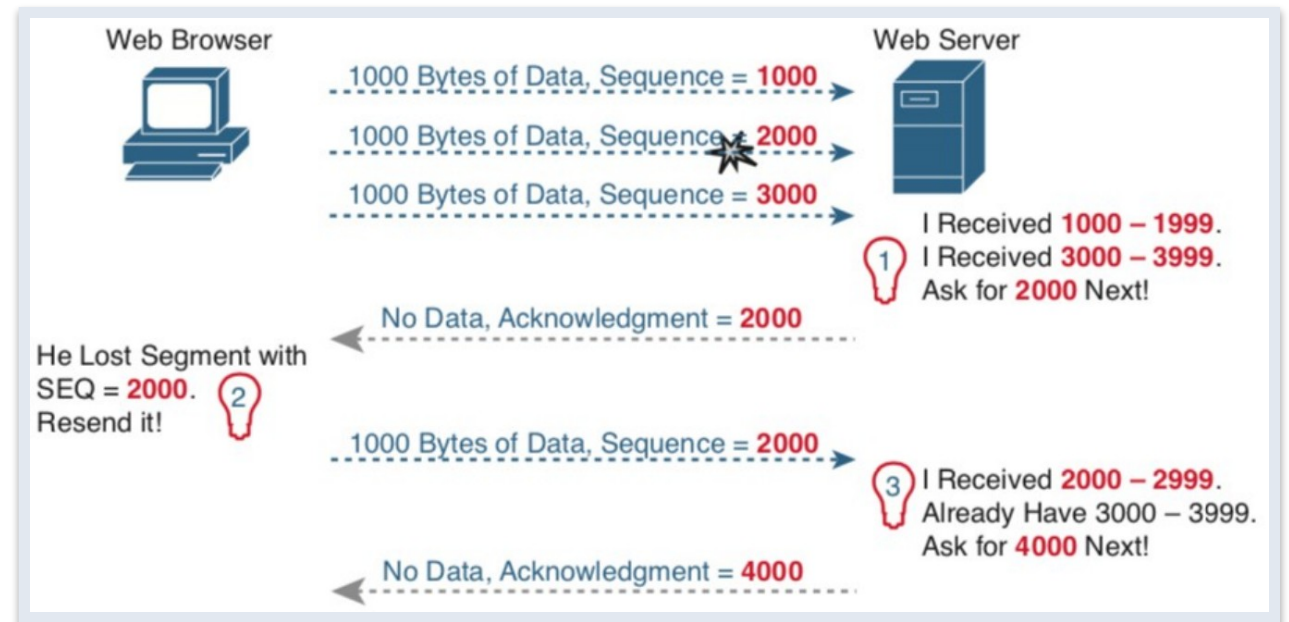
- SYN → SYN/ACK → ACK establishes the connection.
- Options are negotiated here: MSS, window scale, SACK permitted.
- The SYN → SYN/ACK gap is the initial RTT seen by the client.



Connection establishment

Sequence Numbers and Error Recovery

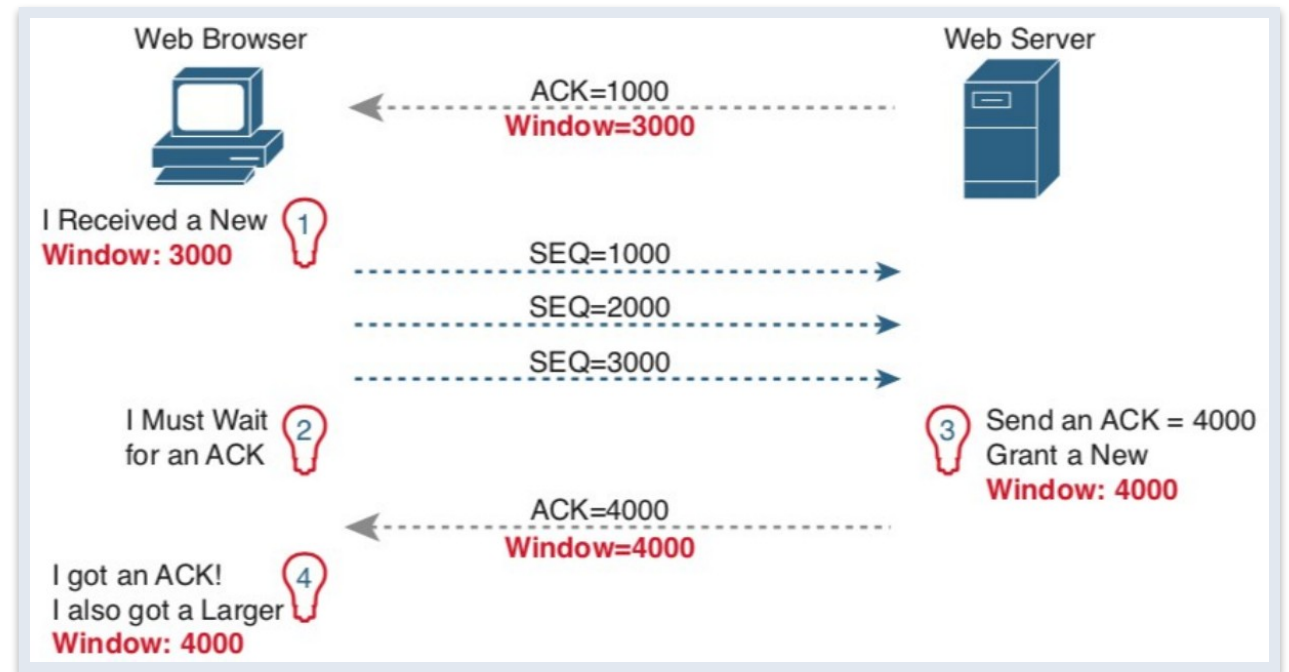
- Sequence numbers count bytes sent; ACK = next byte expected.
- A gap triggers duplicate ACKs and retransmission.
- Wireshark shows relative sequence numbers by default.



Lost segment recovered after the receiver asks again

TCP Windowing

- The receive window advertises buffer space available.
- Window scaling multiplies it for high-bandwidth paths.
- Window = 0 (Zero Window) stops the sender until a window update.

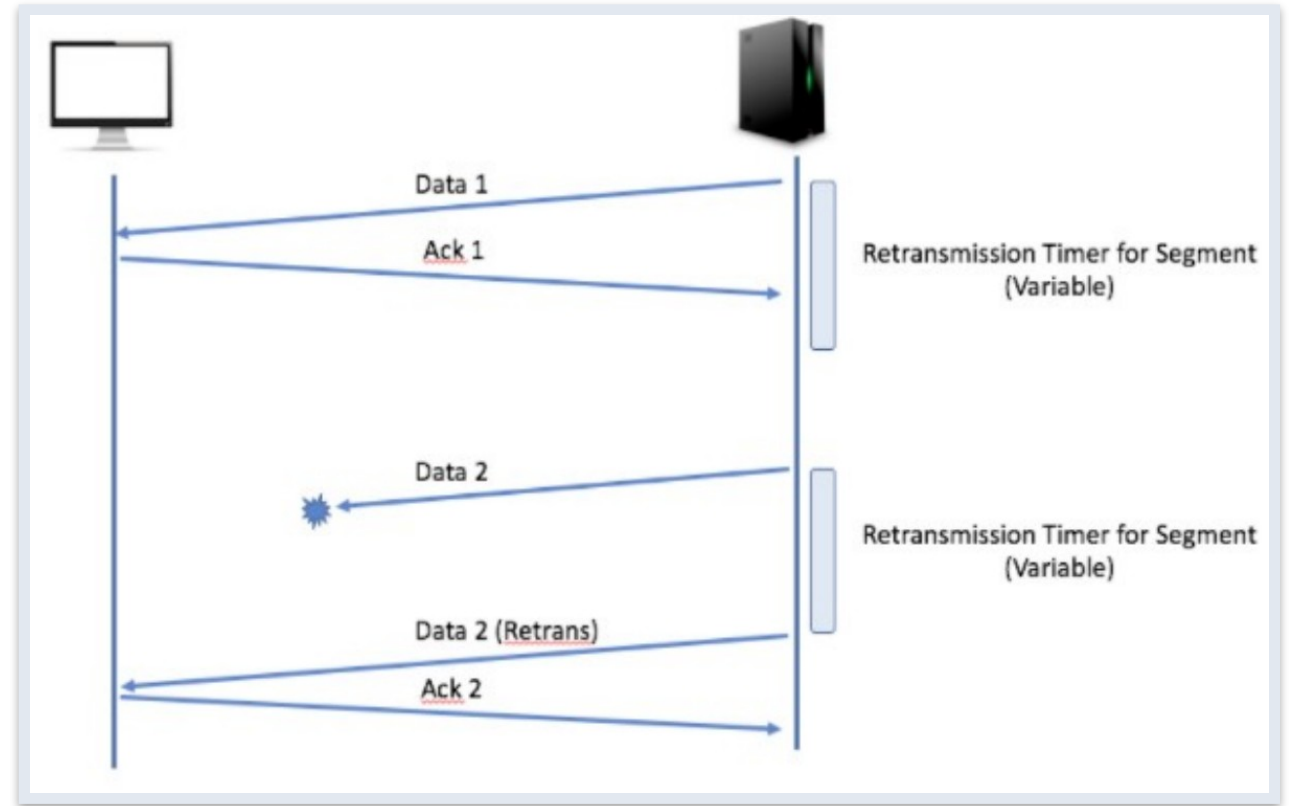


Flow control with window advertisements

LOSS

Retransmissions

- Timeout retransmission (RTO): no ACK in time — backs off exponentially.
- Fast retransmission: three duplicate ACKs trigger an early resend.
- Spurious retransmissions repeat data that was already received.



Retransmission timers on a lossy path

TCP Expert Flags

1

Retransmission

Data sent again

2

Duplicate ACK

Receiver still waiting for a gap

3

Previous segment not captured

Gap — loss or capture drop

4

Zero Window

Receiver buffer full

5

Window Full

Sender filled the advertised window

6

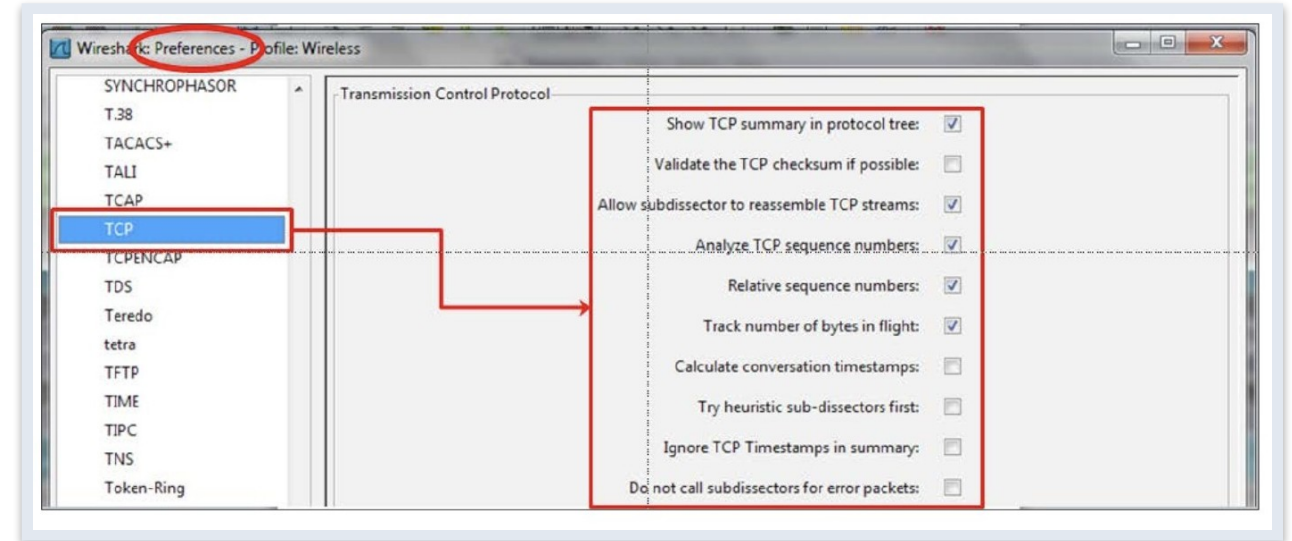
RST

Abrupt connection reset

PREFERENCES

TCP Preferences

- Analyze TCP sequence numbers — enables tcp.analysis.* flags.
- Calculate conversation timestamps — adds tcp.time_delta.
- Relative sequence numbers make streams readable.



Edit | Preferences | Protocols | TCP

Investigate retransmission and zero window

LAB 14

Sequence numbers count bytes; ACK numbers indicate the next expected byte.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 14

tcp.analysis.retransmission

No.	Time	Source	Destination	Protocol	Info
51	3.270000	192.0.2.20	192.0.2.10	TCP	[TCP Retransmission] 80 → 51004 [PSH, ACK] Seq=1 ...
branch-office.pcap Displayed: 1					

tcp.analysis.zero_window

No.	Time	Source	Destination	Protocol	Info
52	3.280000	192.0.2.10	192.0.2.20	TCP	[TCP ZeroWindow] 51004 → 80 [ACK] Seq=70 Ack=93 W...
branch-office.pcap Displayed: 1					

tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Info
56	3.320000	192.0.2.20	192.0.2.10	TCP	81 → 52000 [RST, ACK] Seq=1 Ack=1 Win=8192 Len=0
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Investigate retransmission and zero window

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```


Investigate retransmission and zero window

STEP 2 OF 6

2

Open data/branch-office.pcap. Apply tcp.stream == 3. Open Analyze > Expert Information and note the repeated server segment.

Investigate retransmission and zero window

STEP 3 OF 6

3

Apply `tcp.analysis.retransmission` and expand TCP. Compare the same sequence range and payload in the earlier segment; the repeat occurs 1.000 second later.

Investigate retransmission and zero window

STEP 4 OF 6

4

Apply `tcp.window_size_value == 0`. Confirm the client ACK advertises zero receive window in stream 3. Examine the SYN options: window scaling is negotiated, but zero remains zero.

Investigate retransmission and zero window

STEP 5 OF 6



5

Apply `tcp.flags.reset == 1`. Identify the port 81 refusal. Complete `assets/tcp-evidence.csv` with stream, frame, flag/analysis, measured interval and a limitation. Discuss SACK, fast recovery and out-of-order evidence without claiming those events occur in this fixture.

Investigate retransmission and zero window

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Investigate retransmission and zero window

Test it

The supplied `tcp.analysis.retransmission` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TCP Protocol Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

Sequence numbers count bytes; ACK numbers indicate the next expected byte.

3

Retransmission and duplicate ACK labels are heuristics sensitive to capture placement and completeness.

4

Zero window indicates receive-side flow control; it is distinct from network congestion.

End of Day 3

See you tomorrow

DAY 4

Graphs, applications and reporting

Recap of the previous day, then today's topics and labs

TOPIC 15

Traffic Graphs

Graph bytes, RTT and recovery

Key Concepts — Traffic Graphs

1

An I/O graph bins observed events; units and interval determine what it means.

2

TCP time-sequence graphs show byte progress, stalls and repeated sequence ranges.

3

RTT samples, application response time and throughput answer different questions.

I/O Graphs

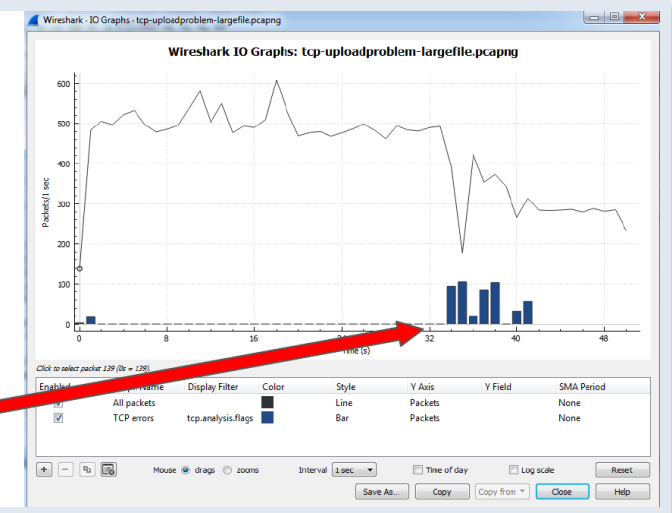
- Statistics > I/O Graphs plots packets, bytes or bits per interval.
- Add graphs with filters — e.g. total vs tcp.analysis.flags.
- Use log scale and moving averages to compare very different values.

SMA → Smooth Moving Average

Log Scale

Use when there is a relatively large difference between the two numerical values and the relationship can be difficult to see.

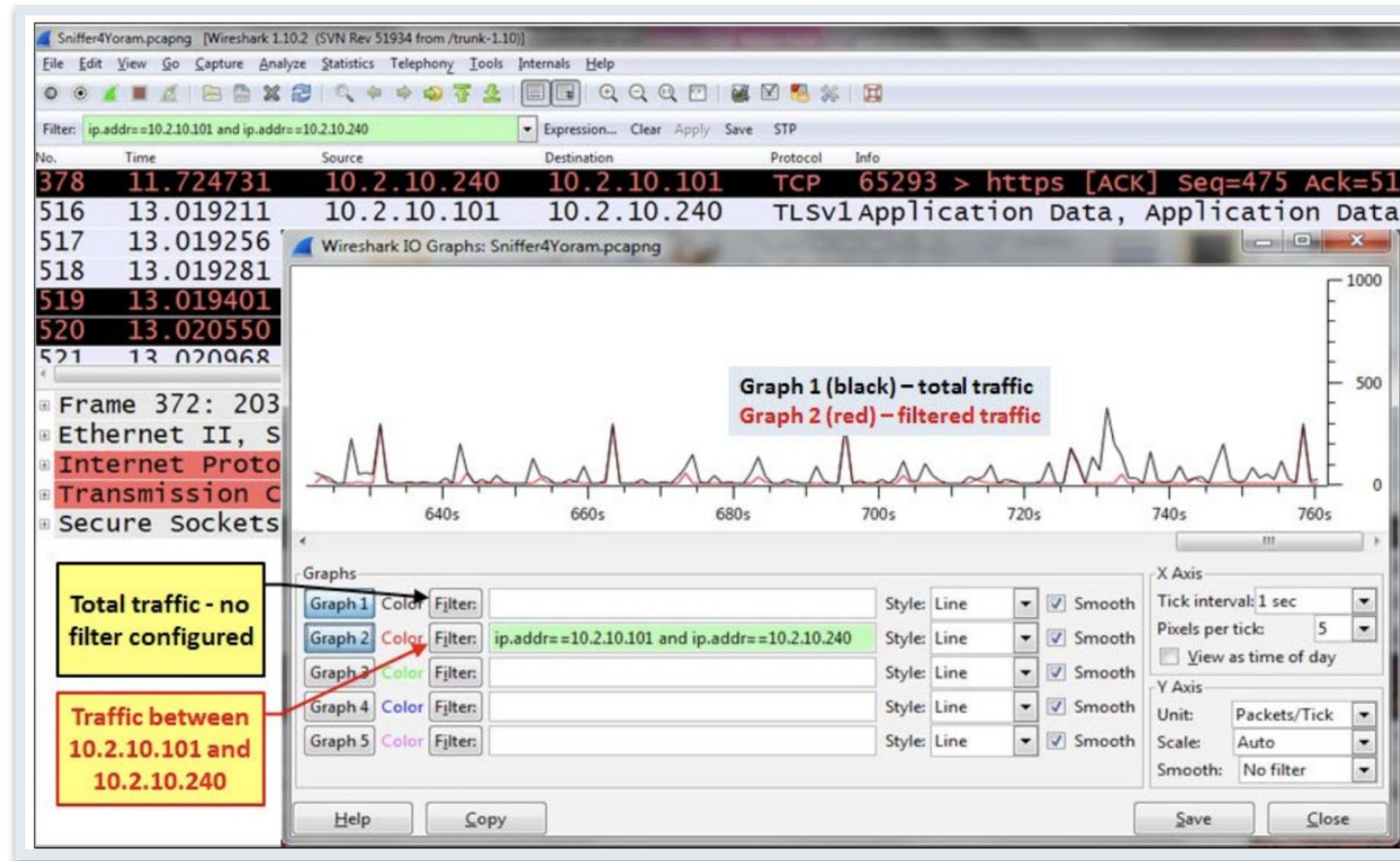
WindowZero, DupAck, etc.



Multiple graphs with different filters on one chart

GRAPHS

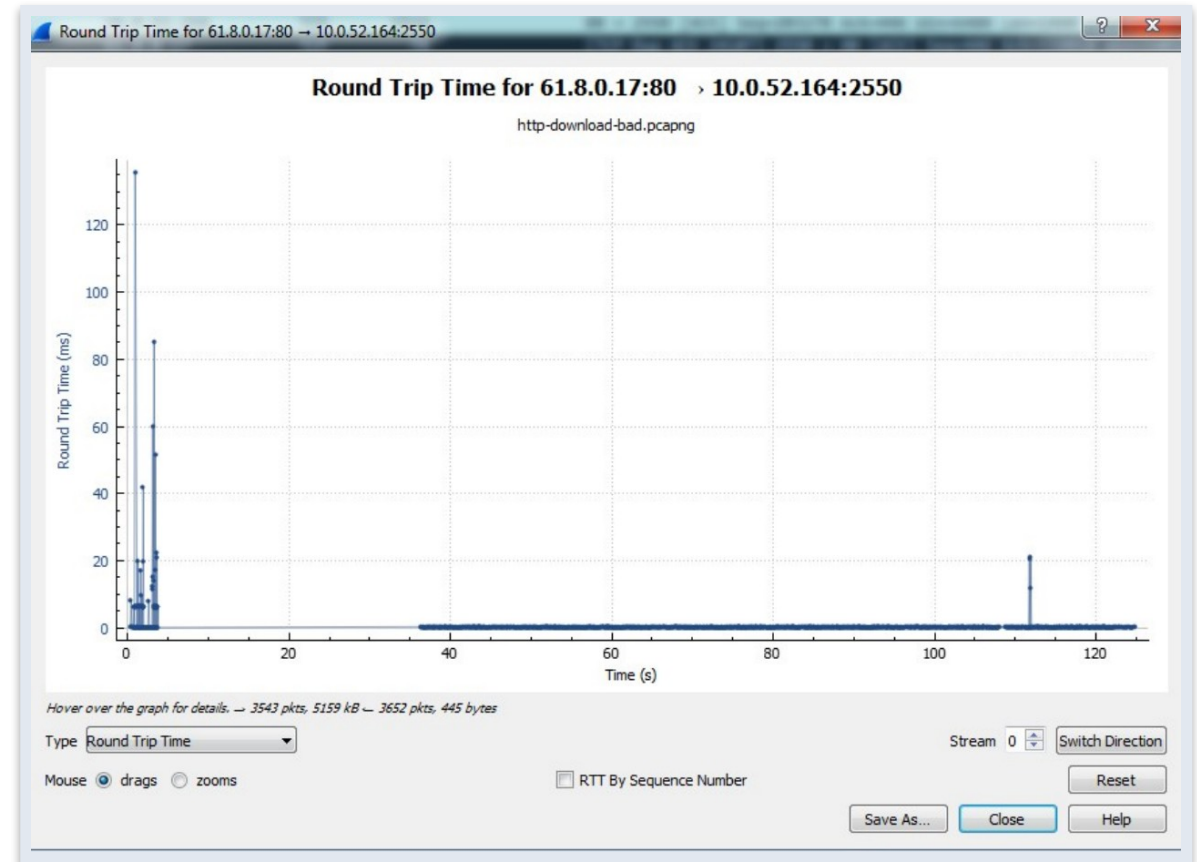
Reading an I/O Graph



Drops in throughput that line up with TCP expert events point to the cause

Round Trip Time Graph

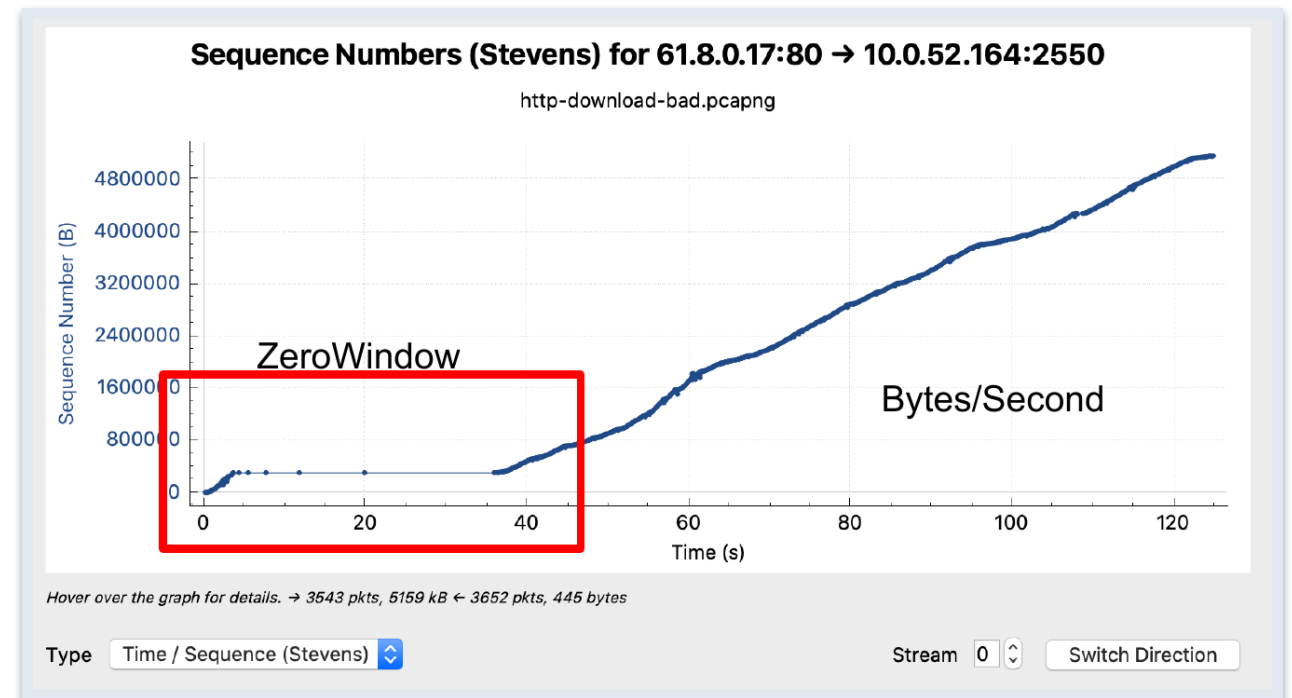
- Statistics > TCP Stream Graphs > Round Trip Time.
- Graphs are unidirectional — pick the data-sending direction.
- Spikes show where the path or receiver was slow to acknowledge.



RTT per acknowledged segment

Time/Sequence (Stevens) Graph

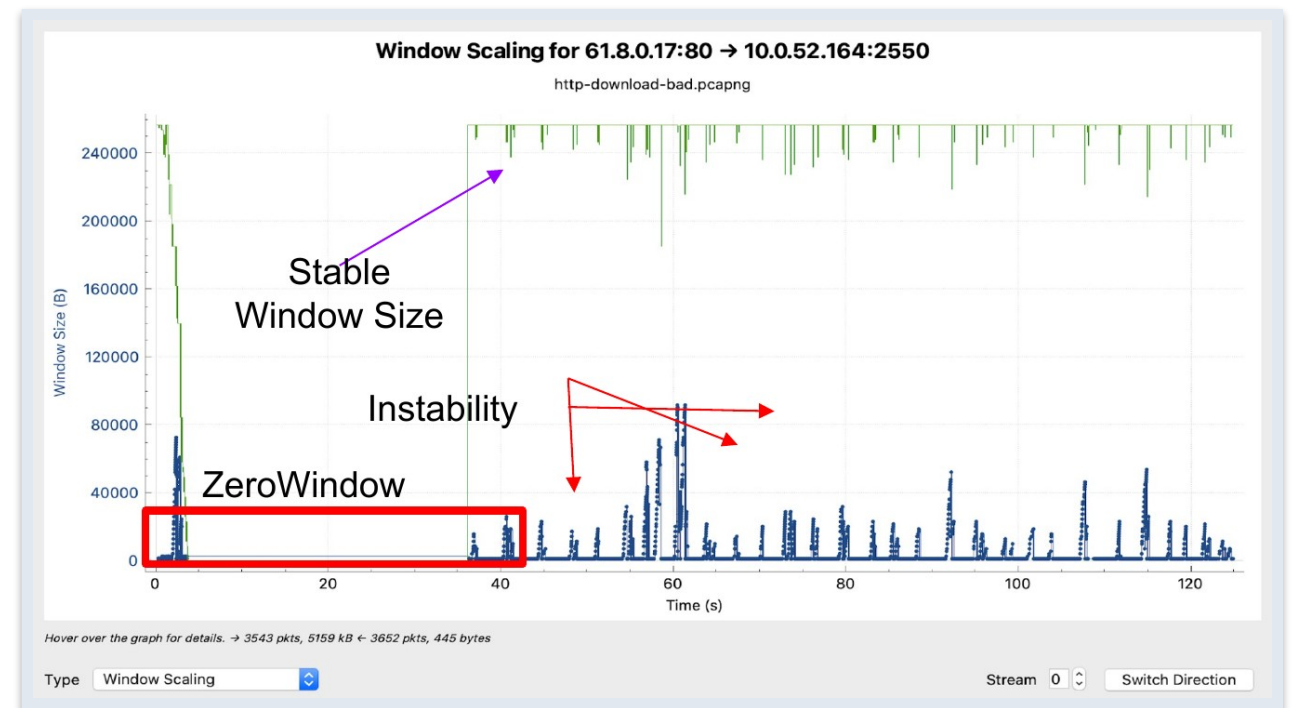
- Sequence number vs time — the slope is throughput.
- Flat steps reveal stalls such as zero window.
- Vertical repeats indicate retransmissions.



A flat region marks a zero-window stall

Window Scaling Graph

- Plots the receiver's advertised window alongside bytes in flight.
- When bytes in flight meet the window, the receiver is the bottleneck.
- A collapsing window points at the receiving application.



Stable window, instability and zero window

Graph bytes, RTT and recovery

LAB 15

An I/O graph bins observed events; units and interval determine what it means.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 15

tcp.analysis.retransmission tcp.analysis.zero_window					
No.	Time	Source	Destination	Protocol	Info
51	3.270000	192.0.2.20	192.0.2.10	TCP	[TCP Retransmission] 80 → 51004 [PSH, ACK] Seq=1 ...
52	3.280000	192.0.2.10	192.0.2.20	TCP	[TCP ZeroWindow] 51004 → 80 [ACK] Seq=70 Ack=93 W...
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Graph bytes, RTT and recovery

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Graph bytes, RTT and recovery

STEP 2 OF 6

2

Open data/branch-office.pcap. Select a packet in TCP stream 3. Open Statistics > TCP Stream Graphs > Time Sequence (Stevens) and inspect the repeated server sequence.

Graph bytes, RTT and recovery

STEP 3 OF 6

3

Open Statistics > I/O Graphs. Add a retransmission graph with filter `tcp.analysis.retransmission` and a zero-window graph with filter `tcp.analysis.zero_window`. Use packets and a 1 second interval; record their event bins.

Graph bytes, RTT and recovery

STEP 4 OF 6

4

Open TCP Stream Graphs > Round Trip Time. Compare eligible ACK samples with the 30 ms SYN-to-SYN/ACK interval and the 40 ms complete handshake/initial_rtt on the final ACK. Explain why no ACK sample exists for an unacknowledged segment before its repeat.

Graph bytes, RTT and recovery

STEP 5 OF 6

5

Run the report script to export timestamps and byte lengths, then write outputs/graph-notes.md with interval, unit, stream, visible pattern and limitation. Do not use SUM(tcp.seq) as throughput.

Graph bytes, RTT and recovery

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```


Graph bytes, RTT and recovery

Test it

The supplied `tcp.analysis.retransmission || tcp.analysis.zero_window` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Traffic Graphs

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

An I/O graph bins observed events; units and interval determine what it means.

3

TCP time-sequence graphs show byte progress, stalls and repeated sequence ranges.

4

RTT samples, application response time and throughput answer different questions.

Tea Break

15 minutes

TOPIC 16

HTTP and HTTP/2 Analysis

Inspect HTTP outcomes and HTTP/2 frames

Key Concepts — HTTP and HTTP/2 Analysis

1

HTTP response codes describe application outcomes after transport delivery.

2

A response time contains more than network latency; pair the correct request and response.

3

HTTP/2 multiplexes frames; encrypted HTTP/2 needs authorised TLS secrets to inspect payload.

HTTP Response Codes

2xx / 3xx

- 200 OK
- 301/302 redirect
- 304 not modified

4xx client

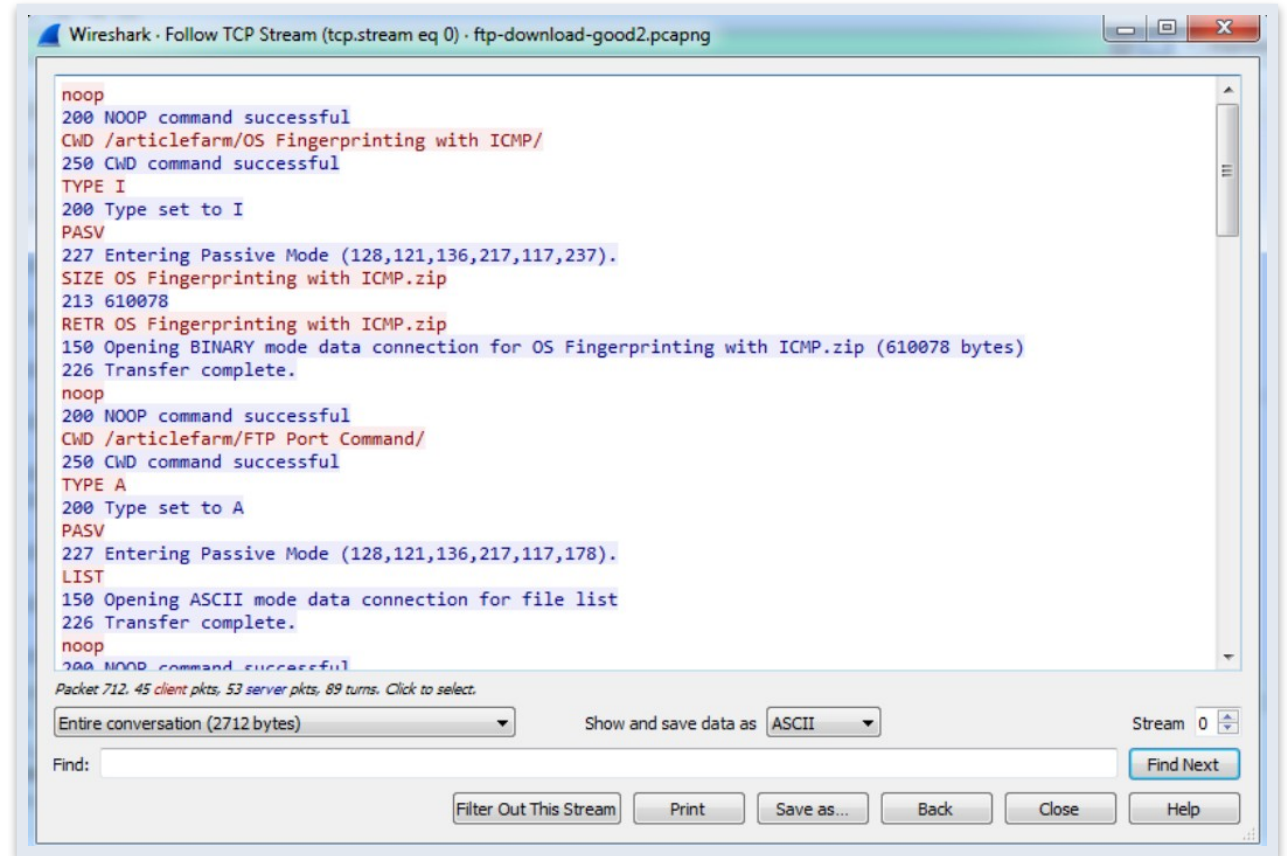
- 400 bad request
- 403 forbidden
- 404 not found

5xx server

- 500 internal error
- 502 bad gateway
- 503 unavailable

Follow the Stream

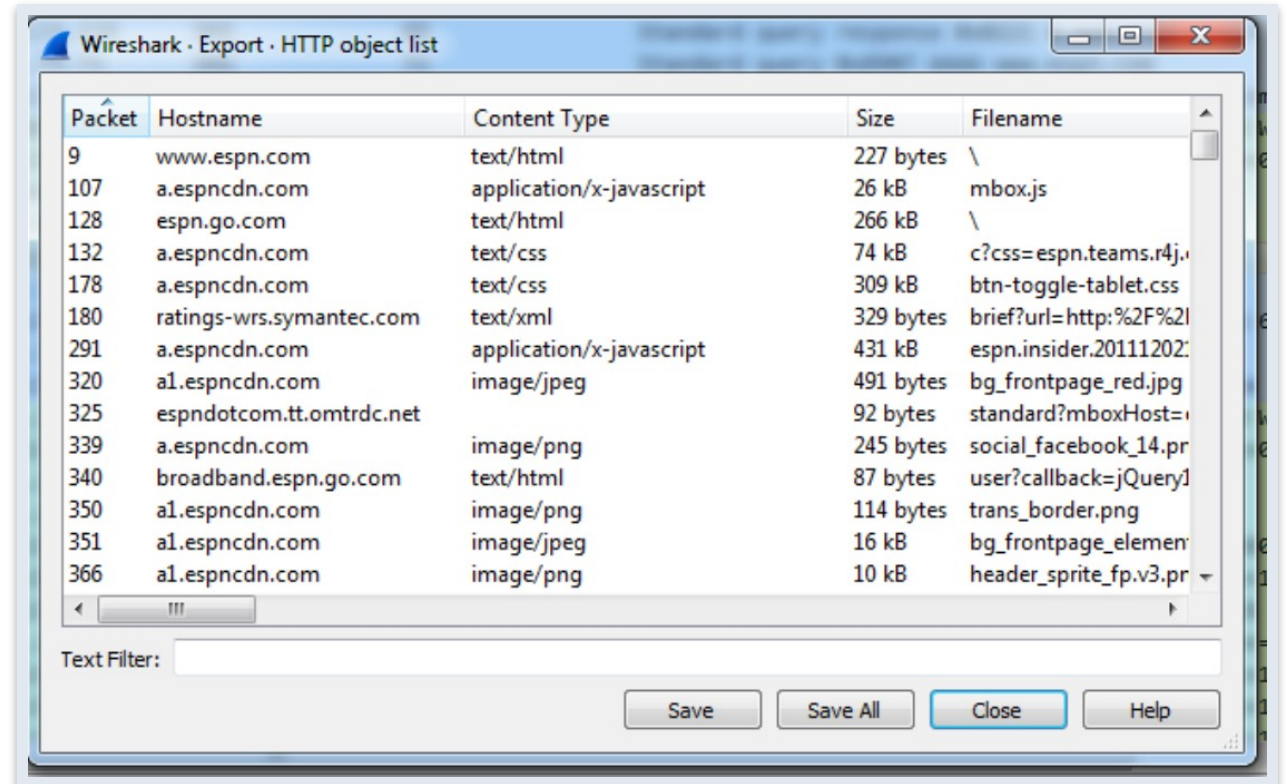
- Right-click > Follow > TCP Stream (or HTTP Stream) reassembles the conversation.
- Client data in red, server data in blue.
- The filter changes to tcp.stream == N — clear it to return.



Follow Stream reassembles the transferred data

Export HTTP Objects

- File > Export Objects > HTTP lists every transferred object.
- Save files for inspection — handle unknown content safely.
- http.time measures each response time.



File | Export Objects | HTTP

HTTP/1.1 vs HTTP/2

HTTP/1.1

- Text headers, one request at a time per connection
- Readable directly in the packet list
- `http.request.method`, `http.response.code`

HTTP/2

- Binary frames multiplexed into streams
- Usually inside TLS (ALPN h2)
- `http2.type`, `http2.streamid`; Decode As for clear-text test ports

Inspect HTTP outcomes and HTTP/2 frames

LAB 16

HTTP response codes describe application outcomes after transport delivery.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 16

http.response.code >= 400					
No.	Time	Source	Destination	Protocol	Info
41	2.130000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 404 Not Found (text/plain)
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap Displayed: 2					
http2.type == 4 [Decode As: tcp.port==8080,http2]					
No.	Time	Source	Destination	Protocol	Info
66	3.460000	192.0.2.10	192.0.2.20	HTTP2	Magic, SETTINGS[0]
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Inspect HTTP outcomes and HTTP/2 frames

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Inspect HTTP outcomes and HTTP/2 frames

STEP 2 OF 6



2

Open data/branch-office.pcap. Apply `http.request || http.response`. Pair `/health`, `/slow`, `/missing` and `/fault` with their response codes.

Inspect HTTP outcomes and HTTP/2 frames

STEP 3 OF 6

3

Select the /health response and Follow > TCP Stream. Confirm LAB-OK. Use File > Export Objects > HTTP to save the health text object into outputs/objects/.

Inspect HTTP outcomes and HTTP/2 frames

STEP 4 OF 6

4

Apply `tcp.port == 8080`. Use Analyze > Decode As and set TCP port 8080 to HTTP2. Inspect the connection preface and SETTINGS frame (type 4). This fixture uses cleartext prior knowledge, not HTTPS.

Inspect HTTP outcomes and HTTP/2 frames

STEP 5 OF 6



5

Run scripts/verify.py, then save outputs/http-summary.csv with URI, status, response interval and proposed next step. Discuss how multiple HTTP/2 stream IDs share one TCP connection.

Inspect HTTP outcomes and HTTP/2 frames

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Inspect HTTP outcomes and HTTP/2 frames

Test it

The supplied `http.response.code >= 400` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — HTTP and HTTP/2 Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

HTTP response codes describe application outcomes after transport delivery.

3

A response time contains more than network latency; pair the correct request and response.

4

HTTP/2 multiplexes frames; encrypted HTTP/2 needs authorised TLS secrets to inspect payload.

Lunch Break

1 hour

TOPIC 17

TLS-Encrypted Traffic Analysis

Compare encrypted and decrypted views

Key Concepts — TLS-Encrypted Traffic Analysis

1

Without session secrets, encrypted application records do not disclose HTTP payload.

2

A controlled key log enables authorised inspection of this synthetic TLS exchange.

3

A server RSA private key alone cannot decrypt modern ECDHE or TLS 1.3 sessions.

TLS Handshake Evidence

Client Hello

- SNI = requested server name
- Offered versions and ciphers
- `tls.handshake.type == 1`

Server Hello

- Chosen version and cipher
- Certificate (TLS 1.2)
- `tls.handshake.type == 2`

Application Data

- Encrypted records
- Sizes and timing still visible
- `tls.record.content_type == 23`

Decrypting TLS — Authorised Only

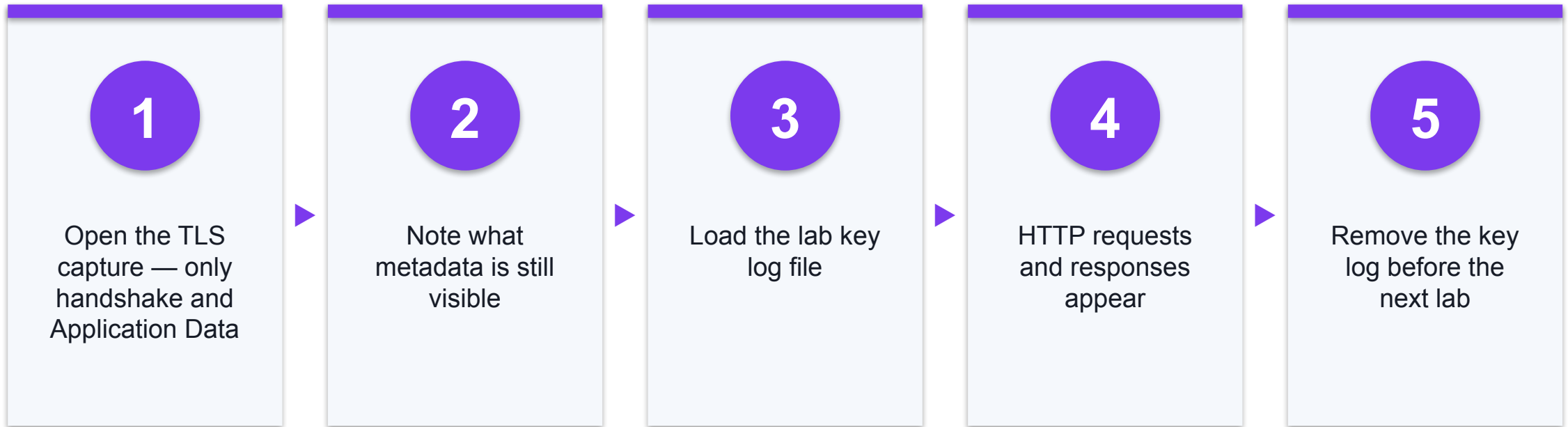
Works

- Key log file from the client (SSLKEYLOGFILE)
- Preferences > Protocols > TLS > (Pre)-Master-Secret log filename
- Works for TLS 1.2 and TLS 1.3

Does not work

- RSA private key with (EC)DHE or TLS 1.3 sessions
- Captures without the matching session secrets
- Anything you are not authorised to inspect

Encrypted vs Decrypted View



Compare encrypted and decrypted views

LAB 17

Without session secrets, encrypted application records do not disclose HTTP payload.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 17

tls.handshake.type == 1					
No.	Time	Source	Destination	Protocol	Info
4	0.060000	192.0.2.10	192.0.2.20	TLSv1	Client Hello (SNI=portal.example.test)
tls-session.pcap Displayed: 1					
http.request					
No.	Time	Source	Destination	Protocol	Info
(no packets match – expected for this view)					
tls-session.pcap Displayed: 0					
http.request [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
12	0.220000	192.0.2.10	192.0.2.20	HTTP	GET /health HTTP/1.1
tls-session.pcap Displayed: 1					
http.response.code == 200 [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
14	0.260000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK
tls-session.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Compare encrypted and decrypted views

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Compare encrypted and decrypted views

STEP 2 OF 6

2

Open data/tls-session.pcap. In Preferences > Protocols > TLS, clear the (Pre)-Master-Secret log filename, then apply tls. Identify ClientHello and encrypted application records.

Compare encrypted and decrypted views

STEP 3 OF 6

3

Expand ClientHello > Extensions > server_name. Record portal.example.test. The lab uses a real TLS 1.2 MemoryBIO exchange, a temporary self-signed certificate and offline synthetic TCP wrapping.

Compare encrypted and decrypted views

STEP 4 OF 6

4

Set Preferences > Protocols > TLS > (Pre)-Master-Secret log filename to the absolute path of data/lab-tls.keys. Reload the capture. Apply http and confirm GET /health and 200 OK containing LAB-OK.

Compare encrypted and decrypted views

STEP 5 OF 6

5

Run scripts/verify.py for both encrypted/decrypted checks. Clear the key-log preference when finished. The supplied secrets are synthetic lab session material; never copy production session secrets into a public repository.

Compare encrypted and decrypted views

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Compare encrypted and decrypted views

Test it

The supplied `tls.handshake.type == 1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TLS-Encrypted Traffic Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

Without session secrets, encrypted application records do not disclose HTTP payload.

3

A controlled key log enables authorised inspection of this synthetic TLS exchange.

4

A server RSA private key alone cannot decrypt modern ECDHE or TLS 1.3 sessions.

Tea Break

15 minutes

TOPIC 18

Ten Troubleshooting Steps and Reporting

Produce an evidence-led incident report

KEY CONCEPTS

Key Concepts — Ten Troubleshooting Steps and Reporting

1

Begin with scope and a baseline, then focus with endpoints, filters, time and graphs.

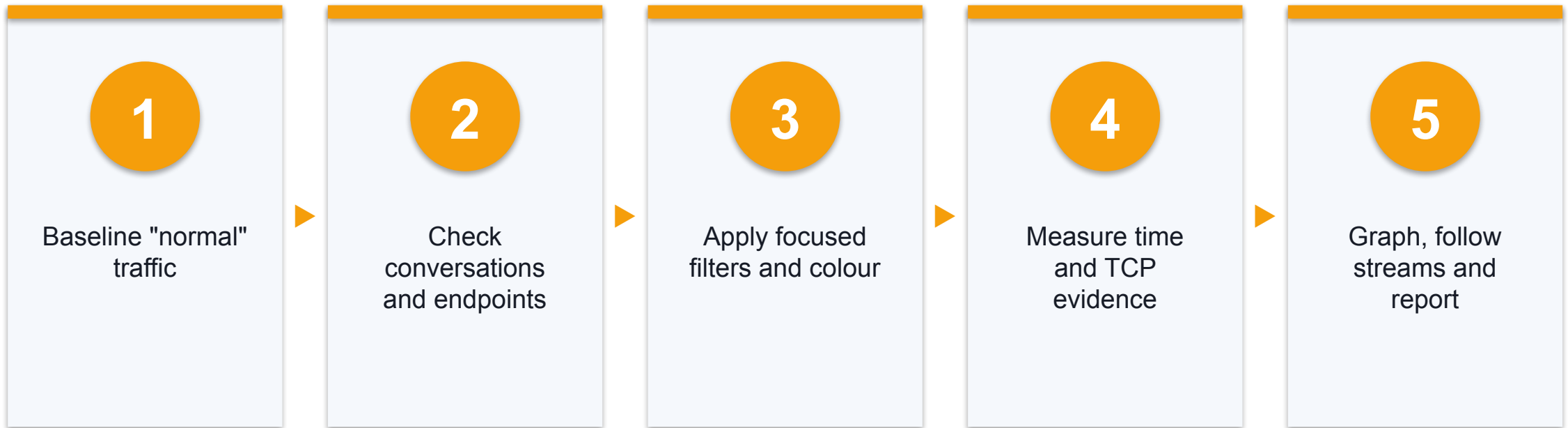
2

Separate observed facts from hypotheses and specify what new evidence would test them.

3

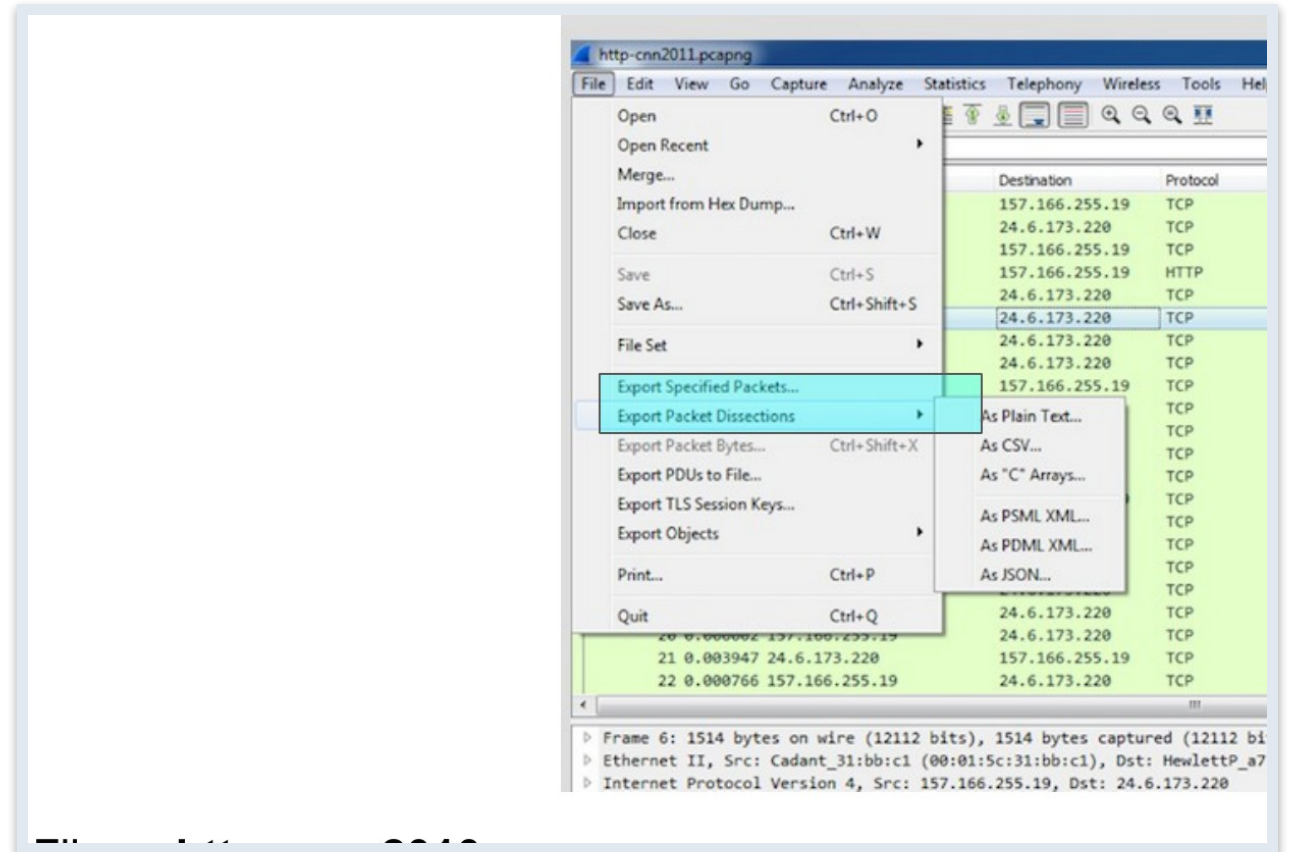
A useful report states affected flow, measurement, impact, recommended action and uncertainty.

Ten-Step Troubleshooting Method



Export and Preserve Evidence

- File > Export Specified Packets saves only the relevant frames.
- Export Packet Dissections produces text/CSV for reports.
- Keep the original capture unchanged and record its hash.



File | Export Packet Dissections

Writing the Incident Report

Observed facts

- Frame numbers
- Filters used
- Measured times and counts

Interpretation

- What the evidence suggests
- Alternative explanations
- Confidence level

Next actions

- Further captures needed
- Owner and system to check
- Limitations of this trace

Produce an evidence-led incident report

LAB 18

Begin with scope and a baseline, then focus with endpoints, filters, time and graphs.

You'll build

Evidence CSV and packet findings

Tools: Wireshark, TShark, Python

Expected Evidence — Lab 18

http.response.code == 500					
No.	Time	Source	Destination	Protocol	Info
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Produce an evidence-led incident report

STEP 1 OF 6

1

Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
$ python3 scripts/verify.py
```

Produce an evidence-led incident report

STEP 2 OF 6

2

Open data/branch-office.pcap. Read assets/incident-ticket.md. Use Statistics > Conversations, filters and Expert Information to identify the failing /fault request.

Produce an evidence-led incident report

STEP 3 OF 6

3

Record evidence for the DNS delay, slow HTTP response, retransmission, zero window and port 81 refusal. Use exact frames and elapsed times rather than a claim that every symptom has one cause.

Produce an evidence-led incident report

STEP 4 OF 6

4

Complete assets/report-template.md and save as outputs/incident-report.md. Include scope, timeline, observations, competing explanations, next capture placement and a client-safe recommendation.

Produce an evidence-led incident report

STEP 5 OF 6

5

Use `assets/ten-step-checklist.md` to review the report. Re-run `scripts/verify.py`. Exchange reports with another learner and check that every conclusion can be traced to a packet or is explicitly a hypothesis.

Produce an evidence-led incident report

STEP 6 OF 6

6

Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in outputs/findings.md.

```
$ python3 scripts/export_evidence.py
```

Produce an evidence-led incident report

Test it

The supplied `http.response.code == 500` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Ten Troubleshooting Steps and Reporting

1

You can now: report observed facts, hypotheses and next actions with defensible evidence.

2

Begin with scope and a baseline, then focus with endpoints, filters, time and graphs.

3

Separate observed facts from hypotheses and specify what new evidence would test them.

4

A useful report states affected flow, measurement, impact, recommended action and uncertainty.



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Capture & profile

Plan scoped captures and configure a reproducible analyst profile.

2

Navigate & time

Navigate, filter, summarize and time network exchanges.

3

Core protocols

Interpret DNS, ARP, IPv4, ICMP and UDP evidence.

4

TCP & applications

Diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

5

Evidence reporting

Report observed facts, hypotheses and next actions with defensible evidence.

Continue Your Learning

1

All course material stays on the LMS at <https://lms-tms.tertiaryinfotech.com>.

2

The 18 labs and their captures are in the course GitHub repository — re-run them any time.

3

Apply the ten-step troubleshooting checklist to an authorised capture at work.

4

Explore networking and cyber-security courses at www.tertiarycourses.com.sg.

Keep Practising After the Course

1

Re-run the labs on your own machine — repetition is what makes the skills stick.

2

Apply the techniques to a real process or project in your own organisation.

3

Your course material stays available at <https://lms-tms.tertiaryinfotech.com/>.

4

Explore the follow-on courses at www.tertiarycourses.com.sg.

THANK YOU FOR ATTENDING

Thank You!

Keep the evidence, explain the uncertainty — and keep practising with the 18 labs on your own machine.